

基于hvisor的飞腾派Type-1 Hypervisor适配方案

一、赛题背景

1.1 信创产业需求

随着“信创”体系的加速推进，国产软硬件在关键领域的应用不断深化，国产处理器平台对虚拟化技术提出了更高的要求。为保障系统的自主可控与运行安全，亟需一套具备高性能、强隔离能力、可裁剪性的虚拟化解决方案。在此背景下，飞腾公司推出的**飞腾派开发板**凭借其对 ARMv8-A 架构的良好支持以及开放的软硬件接口，成为验证国产虚拟化技术的重要平台，尤其适用于教育、工业控制、信息安全等典型场景，具备广阔的应用与推广价值。

1.2 赛题解读与目标

本项目旨在围绕操作系统比赛赛题的技术要求，完成在飞腾平台上部署 Type-1 Hypervisor 的验证性实现，具体目标包括：

1. 在飞腾 CPU 平台上运行 Hypervisor
 - 以裸机方式启动 hvisor，完成对底层平台资源的初始化与管理；
2. 支持同时运行两个及以上虚拟机 (VM)
 - 每个虚拟机运行独立操作系统内核，具备基本 I/O 能力与串口交互；
3. 实现虚拟机生命周期管理接口
 - 提供虚拟机的动态创建、销毁；
 - 支持运行状态控制：启动、暂停、恢复、终止；
 - 提供 CPU 与内存等资源的使用情况监控接口；
4. 实现虚拟机之间的安全隔离与通信机制（可选项）
5. 优化虚拟化系统的性能开销（可选项）
6. 支持虚拟机在飞腾平台间的动态迁移（可选项）

二、系统架构设计

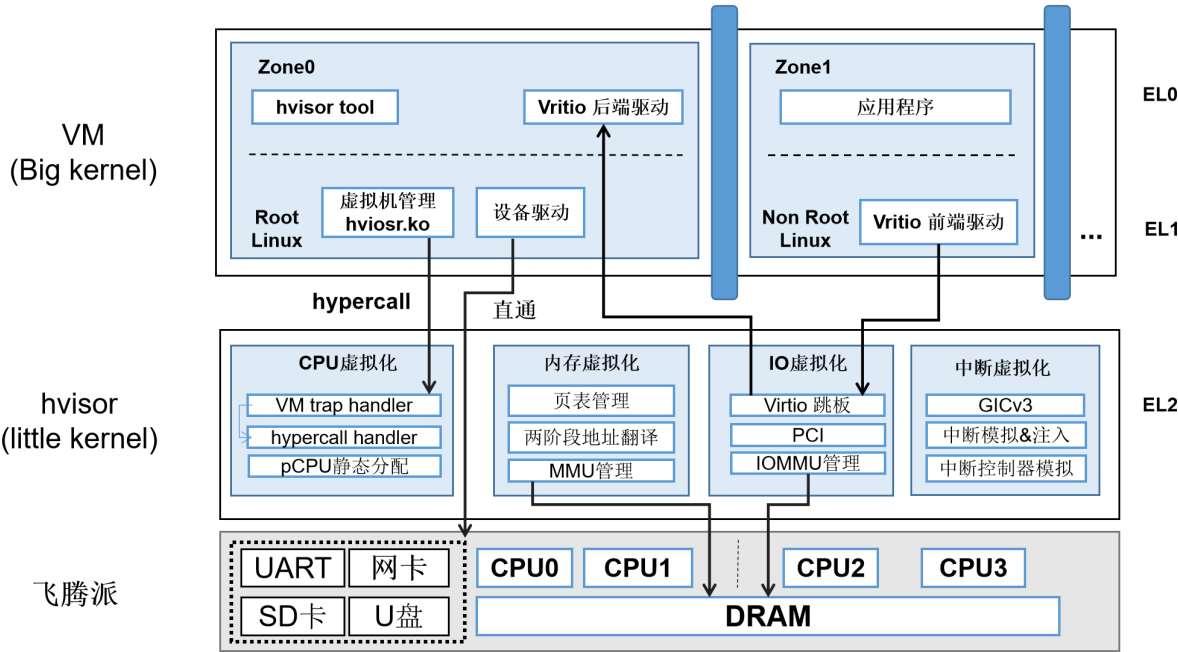
2.1 总体架构图

本项目基于 syswonder 社区的 **hvisor** 实现了飞腾派平台的 Type-1 Hypervisor 适配方案。下图展示了该方案的整体架构。**hvisor** 直接运行在飞腾派开发板上，该平台具备 4 核 ARMv8 架构 CPU、2GB 内存以及串口、网卡、SD 卡和 USB 设备等多种外设资源。作为一个用 Rust 编写的轻量级、高效型 Hypervisor，**hvisor** 在架构中被称为 *Little Kernel*，其核心职责是接管底层硬件资源，并为上层虚拟机提供完整的虚拟化支持，包括 **CPU 虚拟化、内存虚拟化、I/O 虚拟化和中断虚拟化**。在 **hvisor** 提供的虚拟化环境中，可运行多个虚拟机，架构中称为 *Big Kernel*。其中，**zone0** 是 Root 虚拟机，具备对所有虚拟机的管理权限，并通过 Hypercall 接口与 **hvisor** 交互，实现虚拟机的创建、销毁、资源控制等功能。

从异常级（Exception Level）视角来看，**hvisor** 运行于最高特权级 **EL2**，Guest OS（如 non-root Linux）运行在 **EL1**，应用程序运行在 **EL0**。各虚拟化模块设计如下：

- **CPU 虚拟化**：为每个虚拟机提供独立的 CPU 上下文，并注册 trap handler，用于处理来自 Guest 的陷入。采用静态分配的方式划分物理 CPU 资源；
- **内存虚拟化**：利用 ARMv8 硬件支持的两阶段地址转换机制，实现不同虚拟机之间的内存隔离；
- **I/O 虚拟化**：基于 virtio 跳板机制实现，non-root 虚拟机中的 virtio 前端驱动与 zone0 用户态中的 virtio 后端服务通过 `hvisor` 进行通信；同时支持 IOMMU 管理和 PCI 设备模拟；
- **中断虚拟化**：基于 GICv3 架构实现虚拟中断控制器的建模，并支持中断的模拟、注入与转发。

此外，运行在 `zone0` 中的 Root Linux 拥有所有外设的直通访问权限，并通过内核模块配合 `hvisor-tool` 工具，使用 `ioctl` 系统调用与 `hvisor` 通信，实现虚拟机的启动、关闭和资源管理等功能。



2.2 代码结构与平台适配

`hvisor` 为支持多平台设计了模块化配置机制，针对不同平台的配置文件统一存放于 `platform/` 目录下，路径为：

```
/path/to/hvisor/platform/
```

其整体目录结构如下：

```
platform
├── aarch64
│   ├── imx8mp
│   ├── qemu-gicv2
│   ├── qemu-gicv3
│   ├── rk3568
│   ├── ok6254
│   ├── phytium-pi
│   │   ├── board.rs
│   │   ├── cargo
│   │   │   ├── config.template.toml
│   │   │   └── features
│   │   ├── configs
│   │   ├── zone1-linux.json
│   │   └── zone1-linux-virtio.json
│   └── image
```

← 本项目新增平台适配目录
← Zone0 (Root VM) 配置逻辑
← 编译特性配置

```

|   |   |   └─ dts
|   |   |       └─ Makefile
|   |   |       └─ linux1.dts  ← zone0设备树
|   |   |       └─ linux2.dts  ← 精简后的zone1设备树
|   |   |       └─ phytium-pi-board-v2.dts  ← 厂商原始设备树
|   |   └─ linker.ld          ← 镜像链接脚本（指定加载地址）
|   └─ platform.mk            ← 平台编译规则（与 linker.ld 同步）
└─ zcu102
└─ loongarch64
  └─ ls3a5000
    └─ ls3a6000
└─ riscv64
  └─ qemu-aia
    └─ qemu-plic
└─ x86_64
  └─ qemu

```

平台适配说明：

目录结构新增

在 `aarch64/` 目录下新增 `phytium-pi/` 文件夹，即可为飞腾派平台建立独立的启动配置与编译逻辑。

`board.rs`

用于配置启动时 Zone0（Root VM）的参数，包括 CPU 启动顺序、中断布局等平台相关逻辑。

`cargo/features`

用于指定平台所需启用的 Feature 集。必须启用以下核心模块：

- `uart_driver`
- `irqchip_driver`
- `cpu`
- `pt_layout_xx`（xx 为地址布局方案）

`phytium-pi` 的初始 Feature 设置如下：

```

gicv3
phytium_pi
mpidr_phytium

```

- `configs/zone1-linux(-virtio).json`
用于定义 Zone1（非 Root 虚拟机）的内存、CPU、设备映射等信息
- `image/dts/phytium-pi-board-v2.dts`
飞腾派厂商提供的原始设备树文件，需根据 `hvisor` 进行裁剪与修改（如 CPU 节点、直通外设等）
- `linker.ld` & `platform.mk`
 - `linker.ld` 中 `BASE_ADDRESS` 表示 `hvisor.bin` 的加载地址；
 - 此地址需与 U-Boot 中的加载地址保持一致；
 - 若修改了 `BASE_ADDRESS`，应同步修改 `platform.mk` 中相关配置。

 编译注意事项

每次修改 `platform/` 下的配置内容（尤其是设备树、链接地址等）后，**必须执行 `make clean` 再重新编译**，否则构建系统可能会使用缓存的旧配置，导致修改未生效。

2.3 运行步骤

2.3.1 获取系统源码

从百度网盘下载 Phytium 官方系统资源：

- 链接：<https://pan.baidu.com/s/1pStiyqohrB3SxHAFFk8R6Q>
- 提取码：`dzdv`

[返回上一级](#) | [全部文件](#) > [飞腾派资料包](#)（资料包随时更新，... > [4-系统源码](#)

☒ 已选中6个文件/文件夹

☒  GCC编译器

☒  uboot固件

☒  内核设备树

☒  内核源码

☒  phytium-pi sdk使用说明v1.0.pdf

☒  免责声明.pdf

使用到的文件包括：

类型	文件名
交叉编译工具链	<code>gcc-arm-10.2-2020.11-x86_64-aarch64-none-linux-gnu.tar</code>
U-Boot 启动镜像	<code>fip-all-4GB-250321.bin</code>
Linux 内核源码	<code>kernel-5.10.209-phytium-embedded-v2.2.tar.gz</code>
设备树	<code>phytium-pi-board-v2.dtb</code>

2.3.2 linux源码编译

安装aarch64的交叉编译工具

1. 下载并安装交叉编译工具链:

把 gcc-arm-10.2-2020.11-x86_64-aarch64-none-linux-gnu.tar 交叉编译链解压到指定的一个目录, 比如放在 /opt/toolchains/下:

```
$ gcc-arm-10.2-2020.11-x86_64-aarch64-none-linux-gnu ls
10.2-2020.11-x86_64-aarch64-none-linux-gnu-manifest.txt  bin      lib
libexec
aarch64-none-linux-gnu                                include  lib64
share
```

2. 添加路径, 使 aarch64-none-linux-gnu-* 可以直接使用, 修改 ~/.bashrc 文件:

```
echo 'export PATH=$PWD/gcc-arm-10.2-2020.11-x86_64-aarch64-none-linux-
gnu/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

2.3.3 编译linux源码并构建tftp工作目录

1. 解压内核源码并进入目录:

```
tar -xvf kernel-5.10.209-phytium-embedded-v2.2.tar.gz
cd kernel-5.10.209-phytium-embedded-v2.2
```

2. 执行编译命令, 并复制编译后的镜像到 tftp 工作目录:

set_env.sh 设置交叉编译链的环境变量。

```
export PATH=/opt/toolchains/gcc-arm-10.2-2020.11-x86_64-aarch64-none-linux-
gnu/bin:$PATH
export ARCH=arm64 CROSS_COMPILE=aarch64-none-linux-gnu-
export CC=aarch64-none-linux-gnu-gcc
```

build_kernel.sh 编译linux内核并拷贝设备树dtb和linux镜像到tftp目录中。

```
#!/bin/bash
make ARCH=arm64 phytium_defconfig all -j4 INSTALL_MOD_PATH=modules
modules_install
cp ./arch/arm64/boot/dts/phytium/*.dtb /home/b/ft/5.10
cp ./arch/arm64/boot/Image /home/b/ft/5.10
```

这里建立一个tftp目录, 方便之后对镜像整理, 也方便使用tftp传输镜像。

2.3.4 制作sd卡

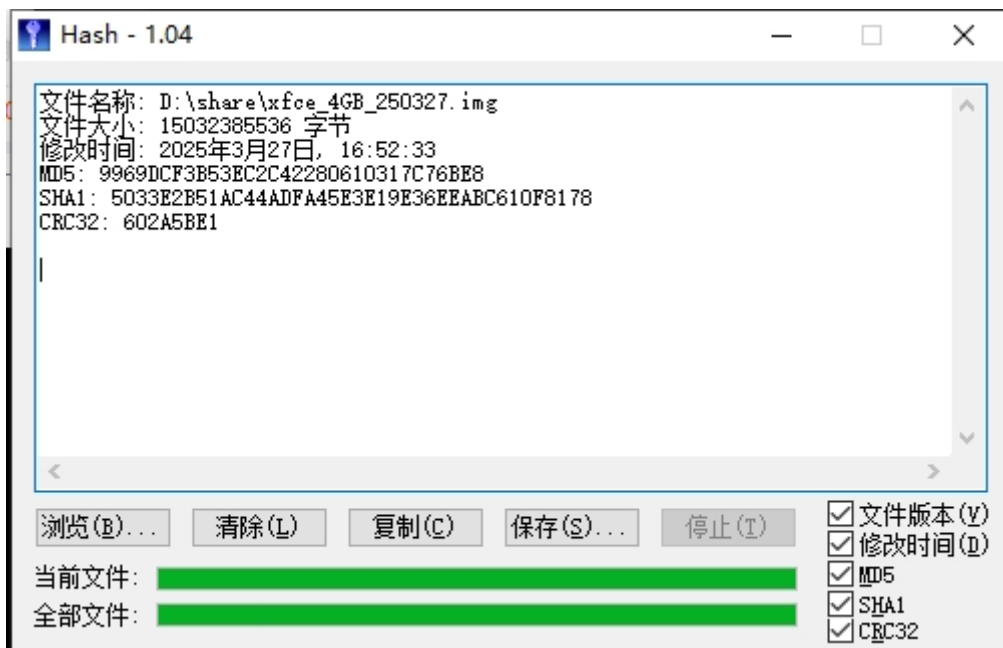
1. 下载官方烧录包:

链接: <https://www.iceasy.com/cloud/Phytium?pid=1914689287345348612>

文件: xfce_4GB_250327.7z

2. 使用 Hash 工具进行镜像烧录, 参考《CEK8903 飞腾派硬件规格书》中的:

- 第 7 章：系统安装与恢复
- 7.1：SD 卡启动方式配置



2.3.5 编译hvisor

进入hvisor目录，切换到**ft分支**，执行编译命令：

```
make BID=aarch64/phytium-pi LOG=info
```

将编译后的**hvisor**镜像放入**tftp**目录

```
make cp
```

2.3.6 启动hvisor和root linux

启动飞腾派开发板之前，将以下文件复制到 SD 卡中的 /home目录：

- Image: root linux镜像，也可以用作non root linux镜像
- linux1.dtb: root linux的设备树
- hvisor.bin: hvisor镜像
- phytium-pi-board-v2.dtb: 这个用于uboot启动时做一些检查，本质没什么用。

启动飞腾派开发板：

1. 调整拨码开关以启用SD卡启动模式。
2. 将SD卡插入SD插槽。
3. 使用串口线将开发板与主机相连。
4. 通过终端软件打开串口。

启动飞腾派板子后，串口应该有输出，重启开发板，立刻按下空格保持不动，使uboot进入命令行终端，执行如下命令：

```

setenv loadaddr 0x90100000;
setenv fdt_addr 0x90000000;
setenv zone0_kernel_addr 0xa0400000;
setenv zone0_fdt_addr 0xa0000000;

ext4load mmc 1:1 ${loadaddr} /home/arm64/hvisor.bin;
ext4load mmc 1:1 ${fdt_addr} /home/arm64/phytium-pi-board-v2.dtb;
ext4load mmc 1:1 ${zone0_kernel_addr} /home/arm64/Image;
ext4load mmc 1:1 ${zone0_fdt_addr} /home/arm64/linux1.dtb;

bootm ${loadaddr} - ${fdt_addr};

```

执行后，hvisor应该就启动并自动进入root linux了。

2.3.7 启动non root linux

启动non root linux需要用到hvisor-tool。

具体请参考[hvisor-tool](#)的 README。

例如，若要编译面向 arm64 的命令行工具，且hvisor 环境中的 Linux 镜像编译来源的源码位于 `~/linux`，则可执行

```
make all ARCH=arm64 LOG=LOG_WARN KDIR=~/linux
```

请务必保证 Hvisor 中的Root Linux 镜像是由编译 hvisor-tool 时参数选项中的 Linux 源码目录编译产生。

编译完成后，将 driver/hvisor.ko、tools/hvisor复制到 SD卡根文件系统中启动 zone1 linux 的目录。

再将 zone1 的内核镜像(Image)、设备树 (linux2.dtb) 放在相同目录。

之后需要为zone1 linux制作一个根文件系统，并改名为 rootfs2.ext4。

下面展示了sd卡中镜像的文件内容。

```

/home/
├─ Image
├─ linux2.dtb
├─ linux1.dtb
├─ phytium-pi-board-v2.dtb
├─ hvisor                -> 命令行工具
├─ hvisor.ko             -> 内核加载模块
├─ rootfs2.ext4          -> non root文件系统镜像
├─ zone1-linux.json      -> non root虚拟机配置文件
└─ zone1-linux-virtio.json -> non root virtio配置文件

```

2.3.8 使用tftp传输镜像

tftp方便开发板与主机间的数据传输，不需要每次插拔sd卡。具体步骤如下：

对于ubuntu系统

如果你使用的是ubuntu系统，则依次执行：

1. 安装 TFTP 服务器软件包

```
sudo apt-get update
sudo apt-get install tftpd-hpa tftp-hpa
```

2. 配置 TFTP 服务器

创建 TFTP 根目录并设置权限：

```
mkdir -p ~/tftp
sudo chown -R $USER:$USER ~/tftp
sudo chmod -R 755 ~/tftp
```

编辑 tftpd-hpa 配置文件：

```
sudo nano /etc/default/tftpd-hpa
```

修改如下：

```
# /etc/default/tftpd-hpa

TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/home/<your-username>/tftp"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="-l -c -s"
```

将 `<your-username>` 替换为实际用户名。

3. 启动/重启 TFTP 服务

```
sudo systemctl restart tftpd-hpa
```

4. 验证 TFTP 服务器

```
echo "TFTP Server Test" > ~/tftp/testfile.txt
```

```
tftp localhost
tftp> get testfile.txt
tftp> quit
cat testfile.txt
```

若显示 "TFTP Server Test"，则 TFTP 服务器工作正常。

5. 配置开机启动：

```
sudo systemctl enable tftpd-hpa
```

6. 使用网线将开发板的网口与主机连接。

并配置主机有线网卡，ip: 192.169.137.1, netmask: 255.255.255.0，启动开发板，进入uboot命令行后，执行命令变为：

```
setenv serverip 192.168.137.1;
setenv ipaddr 192.168.137.2;

setenv loadaddr 0x90100000;
setenv fdt_addr 0x90000000;

setenv zone0_kernel_addr 0xa0400000;
setenv zone0_fdt_addr 0xa0000000;
tftp ${loadaddr} ${serverip}:hvisor.bin;
tftp ${fdt_addr} ${serverip}:phytium-pi-board-v2.dtb;
tftp ${zone0_kernel_addr} ${serverip}:Image;
tftp ${zone0_fdt_addr} ${serverip}:linux1.dtb;
bootm ${loadaddr} - ${fdt_addr};
```

解释：

- `setenv serverip 192.168.137.1`：设置tftp服务器的IP地址。
- `setenv ipaddr 192.169.137.2`：设置开发板的IP地址。
- `setenv loadaddr 0x90100000`：设置hvisor镜像的加载地址。
- `setenv fdt_addr 0x90000000`：设置设备树文件的加载地址。
- `setenv zone0_kernel_addr 0xa0400000`：设置guest Linux镜像的加载地址。
- `setenv zone0_fdt_addr 0xa0000000`：设置root Linux的设备树文件的加载地址。
- `tftp ${loadaddr} ${serverip}:hvisor.bin`：从tftp服务器下载hvisor镜像到hvisor的加载地址。
- `tftp ${fdt_addr} ${serverip}:phytium-pi-board-v2.dtb`：从tftp服务器下载设备树文件到设备树文件的加载地址。
- `tftp ${zone0_kernel_addr} ${serverip}:Image`：从tftp服务器下载guest Linux镜像到guest Linux镜像的加载地址。
- `tftp ${zone0_fdt_addr} ${serverip}:linux1.dtb`：从tftp服务器下载root Linux的设备树文件到root Linux的设备树文件的加载地址。
- `bootm ${loadaddr} - ${fdt_addr}`：启动hvisor，加载hvisor镜像和设备树文件。

重点关注镜像的加载地址是否正确。

三、开发过程与里程碑

3.1 遇到的问题及解决方法

3.1.1 适配飞腾派的加载地址

在尝试通过 U-Boot 阶段执行 TFTP 下载镜像并启动hvisor的过程中，遇到了一个问题：使用 `bootm 0x40400000 - 0x40000000` 命令启动失败，因为地址 `0x40400000 - 0x40000000` 被保护，无法从该地址加载 hvisor：

```
setenv serverip 192.168.137.1; setenv ipaddr 192.168.137.2; setenv loadaddr 0x40400000; setenv fdt_addr 0x40000000; setenv zone0_kernel_addr 0xa0400000; setenv zone0_fdt_addr 0xa0000000; tftp ${loadaddr} ${serverip}:hvisor.bin; tftp ${fdt_addr} ${serverip}:phytium-pi-board-v2.dtb; tftp ${zone0_kernel_addr} ${serverip}:Image; tftp ${zone0_fdt_addr} ${serverip}:linux1.dtb; bootm ${loadaddr} - ${fdt_addr};
```

为了解决上述问题，我们决定直接启动飞腾派定制的 Linux 内核，并进入内核后使用 `bdinfo` 命令查看到飞腾派的 DRAM 起始地址为 `0x80000000`。基于此信息，我们对 hvisor 的链接脚本以及相关 Makefile 进行了修改，以确保 hvisor 可以从正确的内存地址加载，首先修改 hvisor 的链接脚本 `platform/aarch64/phytium-pi/linker.ld`，将基地址设置为 `0x80400000`，以便与新的加载地址相匹配，接下来，需要更新 Makefile 中的相关参数，以反映新的加载地址。具体修改如下：

```
QEMU_ARGS += -device loader,file="$(hvisor_bin)",addr=0x80400000,force-raw=on
QEMU_ARGS += -device loader,file="$(zone0_kernel)",addr=0xa0400000,force-raw=on
QEMU_ARGS += -device loader,file="$(zone0_dtb)",addr=0xa0000000,force-raw=on

QEMU_ARGS += -drive if=none,file=$(FSIMG1),id=xa003e000,format=raw
QEMU_ARGS += -device virtio-blk-device,drive=xa003e000,bus=virtio-mmio-bus.31

$(hvisor_bin): elf
    @if ! command -v mkimage > /dev/null; then \
        sudo apt update && sudo apt install u-boot-tools; \
    fi && \
    $(OBJCOPY) $(hvisor_elf) --strip-all -O binary $(hvisor_bin).tmp && \
    mkimage -n hvisor_img -A arm64 -O linux -C none -T kernel -a 0x80400000 \
    -e 0x80400000 -d $(hvisor_bin).tmp $(hvisor_bin) && \
    rm -rf $(hvisor_bin).tmp
```

3.1.2 实现飞腾派平台的 PL011 串口驱动

在完成 hvisor 的加载地址适配后，我们通过 U-Boot 成功将 hvisor、Linux 内核镜像以及设备树 (DTB) 加载到内存，并尝试启动内核。然而，在执行 `bootm` 命令后，系统卡在了输出 `Starting kernel` ... 阶段，没有任何后续日志输出，导致调试信息无法查看，难以进一步定位问题。

经过对启动流程的分析与排查，最终确认问题是由于 **hvisor 缺乏对飞腾派平台串口驱动的支持** 所致。这导致 hvisor 在启动过程中无法通过串口输出调试信息，进而表现为“卡死”现象，实则程序仍在运行，只是没有控制台输出可供观察。

```
uart@2800d000 {
    compatible = "arm,pl011\0arm,primecell";
    reg = <0x00 0x2800d000 0x00 0x1000>;
    interrupts = <0x00 0x54 0x04>;
    clocks = <0x0c 0x0c>;
    clock-names = "uartclk\0apb_pclk";
    status = "okay";
    phandle = <0x22>;
};
```

从该定义可以看出：

- 飞腾派使用的是 **ARM PL011 UART 控制器**；
- 其寄存器起始物理地址为 `0x2800d000`；

- 支持中断和标准 PL011 功能;
- 串口处于启用状态 (`status = "okay"`) 。

因此, 为了使hvisor能够正常输出调试信息, 需要为其添加对 PL011 串口控制器的支持。在hvisor源码 `src/device/uart/` 目录中新增了一个针对飞腾派平台的串口驱动模块: `phytium_pi.rs`, 内容如下:

```
#![allow(dead_code)]
use tock_registers::interfaces::{Readable, Writeable};
use tock_registers::register_structs;
use tock_registers::registers::{ReadOnly, ReadWrite, WriteOnly};

use crate::memory::addr::{PhysAddr, VirtAddr};
use spin::Mutex;

pub const UART_BASE_PHYS: PhysAddr = 0x2800d000; //phytium_pi uart address

lazy_static! {
    static ref UART: Mutex<P1011Uart> = {
        let mut uart = P1011Uart::new(UART_BASE_PHYS);
        uart.init();
        Mutex::new(uart)
    };
}

register_structs! {
    P1011UartRegs {
        (0x00 => dr: ReadWrite<u32>),
        (0x04 => _reserved0),
        (0x18 => fr: ReadOnly<u32>),
        (0x1c => _reserved1),
        (0x30 => cr: ReadWrite<u32>),
        (0x34 => ifls: ReadWrite<u32>),
        (0x38 => imsc: ReadWrite<u32>),
        (0x3c => ris: ReadOnly<u32>),
        (0x40 => mis: ReadOnly<u32>),
        (0x44 => icr: WriteOnly<u32>),
        (0x48 => @END),
    }
}

struct P1011Uart {
    base_vaddr: VirtAddr,
}

impl P1011Uart {
    const fn new(base_vaddr: VirtAddr) -> Self {
        Self { base_vaddr }
    }

    const fn regs(&self) -> &P1011UartRegs {
        unsafe { &*(self.base_vaddr as *const _) }
    }

    fn init(&mut self) {
        self.regs().icr.set(0x3ff);
    }
}
```

```

        self.regs().ifls.set(0);
        self.regs().imsc.set(1 << 4);
        self.regs().cr.set((1 << 0) | (1 << 8) | (1 << 9));
    }

    fn putchar(&mut self, c: u8) {
        while self.regs().fr.get() & (1 << 5) != 0 {}
        self.regs().dr.set(c as u32)
    }

    fn getchar(&mut self) -> Option<u8> {
        if self.regs().fr.get() & (1 << 4) == 0 {
            Some(self.regs().dr.get() as u8)
        } else {
            None
        }
    }
}

pub fn console_putchar(c: u8) {
    UART.lock().putchar(c)
}

pub fn console_getchar() -> Option<u8> {
    UART.lock().getchar()
}

```

实现了字符的发送与接收功能；加入了并发访问保护机制；在实现了上述 PL011 串口驱动后，重新构建并加载 hvisor，成功看到串口输出的日志信息，不再卡在 `Starting kernel ...` 界面。串口驱动已正确初始化；hvisor 能够正常输出调试信息；

3.1.3 解决hvisor启动时卡在 "Using 4 cpu(s) on this system." 及 CPU2/3 未正常初始化问题

在启动hvisor的过程中，系统卡在了输出 `Using 4 cpu(s) on this system.` 这一行，进一步调试发现程序卡死在 `cpu_start` 函数中。经过详细分析，确认问题出在 PSCI (Power State Coordination Interface) 调用链中的 `psci::cpu_on` 函数，而根本原因在于未能正确获取到多核 CPU 的唯一标识符 `cpuid`。

```

Bytes transferred = 29250 (7242 hex)
## Booting kernel from Legacy Image at 80400000 ...
  Image Name:   hvisor_img
  Image Type:   AArch64 Linux Kernel Image (uncompressed)
  Data Size:    671824 Bytes = 656.1 KiB
  Load Address: 80400000
  Entry Point:  80400000
  Verifying Checksum ... OK
## Flattened Device Tree blob at 80000000
  Booting using the fdt blob at 0x80000000
  Loading Kernel Image
  Loading Device Tree to 00000000f9c27000, end 00000000f9c31a5d ... OK
run in ft_board_setup
fdt_addr 00000000f9c27000
mb_count = 0x2
mb_blocks[0].mb_size = 0x7c000000
mb_blocks[1].mb_size = 0x80000000
fdt : can not find /memory@01 node
fdt : add node memory@01
fdt : dram size 0x100000000 update successfully

Starting kernel ...

Hello, start HVISOR at 0x80400000!
Heap allocator initialization finished: 0x804a7010..0x805a7010
heap_test passed!
Booting CPU 0: 0x805ca000 arch:0x805ca018, DTB: 0xf9c27000
Using 4 cpu(s) on this system.

```

通过查看设备树信息，我们确认飞腾派平台的 CPU 节点确实使用了 `psci` 作为启用方式，并且其底层依赖 SMC 指令与固件通信来唤醒 CPU。然而，飞腾派的 CPU ID 编码方式不同于常见的 ARM 平台，其四颗核心对应的 MPIDR 值分别为 `0x200`, `0x201`, `0x00`, `0x100`，而不是标准的 `0x01`, `0x02`, `0x03`, `0x04`。这导致默认的 CPU ID 映射逻辑无法正确识别这些值，从而引发唤醒失败或后续执行异常。

```

cpu@0 {
    device_type = "cpu";
    compatible = "phytium,ftc310\0arm,armv8";
    reg = <0x00 0x200>;
    enable-method = "psci";
    clocks = <0x09 0x02>;
    capacity-dmips-mhz = <0xb22>;
    phandle = <0x07>;
};

cpu@1 {
    device_type = "cpu";
    compatible = "phytium,ftc310\0arm,armv8";
    reg = <0x00 0x201>;
    enable-method = "psci";
    clocks = <0x09 0x02>;
    capacity-dmips-mhz = <0xb22>;
    phandle = <0x08>;
};

cpu@100 {
    device_type = "cpu";
    compatible = "phytium,ftc664\0arm,armv8";
    reg = <0x00 0x00>;
    enable-method = "psci";
    clocks = <0x09 0x00>;
    capacity-dmips-mhz = <0x161c>;
};

```

```

        phandle = <0x05>;
    };

    cpu@101 {
        device_type = "cpu";
        compatible = "phytium,ftc664\0arm,armv8";
        reg = <0x00 0x100>;
        enable-method = "psci";
        clocks = <0x09 0x01>;
        capacity-dmips-mhz = <0x161c>;
        phandle = <0x06>;
    };

    psci {
        compatible = "arm,psci-1.0";
        method = "smc";
        cpu_suspend = <0xc4000001>;
        cpu_off = <0x84000002>;
        cpu_on = <0xc4000003>;
        sys_poweroff = <0x84000008>;
        sys_reset = <0x84000009>;
    };
};

```

为此，我们在 `cpu_start` 函数中新增了 `mpidr_phytium` 特性标志，用于适配飞腾平台特有的 MPIDR 映射规则。根据 cpuid (0~3) 手动将其转换为对应的核心地址：CPU0 对应 `0x200`，CPU1 对应 `0x201`，CPU2 对应 `0x00`，CPU3 对应 `0x100`。这样确保了每个 CPU 都能被正确唤醒并跳转至指定的入口地址执行初始化代码。

```

pub fn cpu_start(cpuid: usize, start_addr: usize, opaque: usize) {
    if cfg!(feature = "mpidr_phytium") {
        let new_cpuid = match cpuid {
            1 => {
                0x201
            }
            2 => {
                0x00
            }
            3 => {
                0x100
            }
            _ => {
                panic!("Invalid cpuid: {}", cpuid);
            }
        };
        psci::cpu_on(new_cpuid as u64, start_addr as _, opaque as _)
            .unwrap_or_else(|err| {
                println!("psci cpu_on failed: {:?}", err);
                if let psci::error::Error::AlreadyOn = err {
                } else {
                    panic!("can't wake up cpu {}", cpuid);
                }
            })
    } else {

```

```

        psci::cpu_on(cpuid as u64 | 0x80000000, start_addr as _, opaque as
_).unwrap_or_else(|err| {
            println!("psci cpu_on failed: {:?}", err);
            if let psci::error::Error::AlreadyOn = err {
            } else {
                panic!("can't wake up cpu {}", cpuid);
            }
        })
    }
}

```

然而，在实现上述逻辑后，虽然 CPU0 和 CPU1 能够顺利进入 `rust_main` 函数完成初始化流程，但 CPU2 和 CPU3 却始终没有执行任何用户级代码。通过对 `entry.rs` 文件的深入排查，发现问题出在 `boot_cpuid_get()` 函数中——该函数由一段汇编代码实现，负责从 `mpidr_el1` 寄存器中读取当前 CPU 的 ID 值。

原始实现中，该函数仅提取了 `mpidr_el1` 的低 8 位（即 `x17 & #0xff`），这种做法适用于大多数通用平台，但在飞腾派上却存在严重偏差。由于飞腾派使用的 MPIDR 是非标准的 16 位编码方式，CPU2 和 CPU3 的 MPIDR 分别为 `0x00` 和 `0x100`，而这两个值在与 `#0xff` 做 AND 操作后都变成了 `0x00`，最终都被错误地解析为 `cpuid=0`，也就是主核 CPU0 的 ID。这一错误导致 CPU2 和 CPU3 在初始化阶段共享了 CPU0 的栈空间，从而引发了严重的栈冲突和执行异常，表现为它们虽然成功被唤醒，但并未进入 `rust_main` 执行任何初始化动作。

```

pub unsafe extern "C" fn boot_cpuid_get() {
    core::arch::asm!(
        "
        mrs x17, mpidr_el1
        and x17, x17, #0xff
        ret
    ",
        options(noreturn)
    )
}

```

为了彻底解决这个问题，我们重新实现了 `boot_cpuid_get` 的汇编逻辑，新增了对 `mpidr_phytium` 特性的支持，将 MPIDR 的低 16 位全部保留，并通过条件判断明确区分四个不同的核心，分别返回正确的 `cpuid` 值（0、1、2、3）。这样就确保了每个 CPU 都拥有独立的栈空间，避免了资源冲突。

```

#[cfg(feature = "mpidr_phytium")]
#[naked]
#[no_mangle]
pub unsafe extern "C" fn boot_cpuid_get() {
    core::arch::asm!(
        "
        mrs x17, mpidr_el1
        and x17, x17, #0xffff    // low 16位

        cmp x17, #0x200          // cpu0
        b.eq 3f
        cmp x17, #0x201          // cpu1
        b.eq 4f
    "
    )
}

```

```

        // 再检查其他核心
        cmp x17, #0x00          // cpu2
        b.eq 1f
        cmp x17, #0x100        // cpu3
        b.eq 2f

        mov x17, #-1           // unknown CPU
        b 5f

1:  mov x17, #2; b 5f          // back 2
2:  mov x17, #3; b 5f          // back 3
3:  mov x17, #0; b 5f          // back 0
4:  mov x17, #1               // back 1
5:  ret
    ",
    options(noreturn)
);
}

```

与此同时，我们还在 `mpidr_to_cpuid` 函数中加入了对 `mpidr_phytium` 的特性支持。该函数的作用是在系统运行期间根据任意给定的 MPIDR 值反向解析出对应的 CPU ID。通过匹配 `(mpidr & 0xffff)` 的值，我们能够准确地将 `0x00`、`0x100`、`0x200`、`0x201` 四个 MPIDR 值映射为 `cpuid=2`、`3`、`0`、`1`，从而保证整个系统中对 CPU ID 的统一性和一致性。

```

pub fn mpidr_to_cpuid(mpidr: u64) -> u64 {
    #[cfg(feature = "mpidr_rockchip")]
    {
        (mpidr >> 8) & 0xff
    }

    #[cfg(feature = "mpidr_phytium")]
    {
        match (mpidr & 0xffff) as u32 {
            0x00 => 2,
            0x100 => 3,
            0x200 => 0,
            0x201 => 1,
            _ => panic!("Unknown MPIDR: {:#x}", mpidr),
        }
    }

    #[cfg(not(any(feature = "mpidr_rockchip", feature = "mpidr_phytium")))]
    {
        mpidr & 0xff00ffffff
    }
}

```

此外，针对 GICv3 中断控制器的支持，我们也为飞腾平台适配了相应的 MPIDR 到物理 CPU ID 的映射逻辑，以确保中断路由和目标选择的准确性。

综上所述，本次修复主要集中在两个方面：

1. **修正了 CPU 启动时的 MPIDR 到 cpuid 的映射逻辑**，确保每个 CPU 都能被正确唤醒并分配独立的栈空间；
2. **完善了系统运行期的 MPIDR 到 cpuid 的解析机制**，使得包括中断控制在内的多个模块都能准确识别当前运行的 CPU 核心。

3.1.4 earlycon未启导致Linux初始化日志缺失问题

在完成hvisor的串口驱动适配后，成功恢复了控制台输出功能，但此时仍然存在一个问题：**看不到Linux 内核启动过程中日志输出**，在早期内核初始化阶段，如 GIC、时钟、调度器等模块的调试信息未能及时打印出来。通过分析设备树配置和内核启动流程，发现问题是由于 **缺少 earlycon 支持** 所致。为此，我们在设备树中的 `chosen` 节点中添加并启用了如下参数：

```
chosen {
    stdout-path = "serial1:115200n8";
    // bootargs = "console=ttyAMA1,115200 earlyprintk root=/dev/mmcb1k0p1
    rw rootwait";
    bootargs = "clk_ignore_unused console=ttyAMA1,115200
    earlycon=p1011,0x2800d000,115200 root=/dev/mmcb1k0p1 rootwait rw";
};
```

启用了 `earlycon=p1011,0x2800d000`

- 该参数是 **earlycon 驱动的标准格式**，明确告诉内核使用 `p1011` 类型串口，并指定其 MMIO 地址。
- 内核在非常早的阶段（甚至还没初始化 `printk`）时，就可以输出日志 —— 这对于早期内核挂死、卡在 PSCI 或 GIC 等初始化流程的情况 **非常关键**。
- 原本的 `earlyprintk` 方式依赖于编译时默认平台和串口配置。

通过这一修改，终于看到了内核最早启动阶段开始的部分日志输出，为后续调试提供了强有力的支撑。

3.1.5 GICv3 ITS 地址分配冲突导致系统卡死

这是整个 hvisor 移植过程中很隐蔽、很难定位的一个 bug，耗费了整整三天时间才得以解决。最初怀疑是时钟中断注入失败，但在调试过程中发现 Hypervisor 并没有进入任何中断处理函数，也没有接收到任何中断信号。更令人困惑的是，在将定时器频率从 50MHz 提升到 100MHz 后，内核竟然能够完整打印出以下日志：[0.000000] arch_timer: cp15 timer(s) running at 50.00MHz (virt)。但依然卡死在这里，随后继续尝试将时钟频率拉高至 1200MHz，发现还能多输出两条原本被卡住的日志，这进一步表明问题并不在于时钟本身，而是其他更底层机制导致了系统卡死。接着我又猜测是因为 gicv3 初始化失败，hypervisor 的 mmu 没有把 time 相关寄存器映射成 device 类型等等，发现都不是导致卡死的原因，在反复查看日志输出的过程中，突然注意到以下内容：

```
[ 0.000000] GICv3: 256 SPIs implemented
[ 0.000000] GICv3: 0 Extended SPIs implemented
[ 0.000000] GICv3: Distributor has no Range Selector support
[ 0.000000] GICv3: 16 PPIs implemented
[ 0.000000] GICv3: CPU0: found redistributor 200 region 0:0x00000000308c0000
[ 0.000000] ITS [mem 0x30820000-0x3083ffff]
[ 0.000000] ITS@0x0000000030820000: allocated 65536 Devices @80480000 (flat,
esz 8, psz 64K, shr 0)
[ 0.000000] ITS: using cache flushing for cmd queue
[ 0.000000] GICv3: using LPI property table @0x0000000080430000
[ 0.000000] GIC: using cache flushing for LPI property table
[ 0.000000] GICv3: CPU0: using allocated LPI pending table
@0x0000000080440000
[ 0.000000] arch_timer: cp15 timer(s) runn
```

发现GICv3给 ITS 子系统自动分配的地址是地址分别为@80480000、@0x0000000080430000和@0x0000000080440000！！然而我的hvisor.bin的加载地址确是0x80400000（大小约 700KB ≈ 0x000B0000），这些区域覆盖了 hvisor.bin 占用的内存空间，导致 Linux 在初始化 GICv3 ITS 时：引发 Hypervisor 崩溃 或 VGIC 初始化失败；导致中断不触发，pending_irq() 永远返回 None；arch_timer 输出“runni”后卡死，只是中断控制器死锁的表面现象。

根据飞腾派提供的 U-Boot 启动参数，Linux 内核镜像通常加载在 0x90100000，设备树则加载在 0x90000000。因此，为了避免与 Linux 及其子系统的内存分配发生冲突，我们将hvisor的加载地址也调整为 0x90100000，确保其不会侵占后续 Linux 系统使用的内存空间。

```
//platform.mk修改链接脚本和构建脚本
QEMU_ARGS += -device loader,file="$(hvisor_bin)",addr=0x90100000,force-rw=on
$(hvisor_bin): elf
    @if ! command -v mkimage > /dev/null; then \
        sudo apt update && sudo apt install u-boot-tools; \
    fi && \
    $(OBJCOPY) $(hvisor_elf) --strip-all -O binary $(hvisor_bin).tmp && \
    mkimage -n hvisor_img -A arm64 -O linux -C none -T kernel -a 0x90100000 \
    -e 0x90100000 -d $(hvisor_bin).tmp $(hvisor_bin) && \
    rm -rf $(hvisor_bin).tmp
//更新 U-Boot 启动命令
setenv serverip 192.168.137.1; setenv ipaddr 192.168.137.2; setenv loadaddr
0x90100000; setenv fdt_addr 0x90000000; setenv zone0_kernel_addr 0xa0400000;
setenv zone0_fdt_addr 0xa0000000;
tftp ${loadaddr} ${serverip}:hvisor.bin; tftp ${fdt_addr} ${serverip}:phytium-
pi-board-v2.dtb; tftp ${zone0_kernel_addr} ${serverip}:Image; tftp
${zone0_fdt_addr} ${serverip}:linux1.dtb; bootm ${loadaddr} - ${fdt_addr};
```

此次问题的根本原因是 hvisor 的加载地址与 Linux 内核在初始化 GICv3 ITS 时自动分配的地址发生冲突，导致 Hypervisor 被覆盖、中断失效、系统卡死。

3.1.6 unhandled MMIO 异常的原因与处理方法

在hvisor的运行过程中，系统频繁出现 unhandled mmio 错误提示。这类错误通常表明：某个设备的 MMIO 地址空间被访问了，但 Hypervisor 并未对其进行映射或处理。这会导致虚拟机中的设备访问失败，引发系统异常。

经过对日志和设备树的深入分析，我们确认这些错误来源于 **飞腾派平台设备树中定义的一些外设地址未在hvisor中配置对应的内存映射区域**。也就是说，当 Linux 内核尝试访问某些硬件寄存器时，hvisor 因为没有为其建立 MMIO 映射而无法正确响应，从而触发 `unhandled mmio` 异常。

解决步骤：

- 通过日志中报错的地址反推是哪个设备被访问；
- 查看该设备在设备树中的 `reg` 地址范围；
- 在 hvisor 的配置文件 `board.rs` 中，将这些地址添加到 `ROOT_ZONE_MEMORY_REGIONS` 列表中，确保其被正确映射；

禁用不必要的外设

- 对于一些当前并不使用的设备（如 USB、pmdk_dp、dc、pcie、hda、sata、vpu 等），可以直接将其设备节点的 `status` 属性设置为 `"disabled"`，以避免 Linux 内核尝试访问它们；
- 这样可以减少不必要的 MMIO 访问，降低 Hypervisor 负载，同时提升系统的稳定性；

3.1.7 serial-getty 服务异常导致系统假死问题

在解决多个 `unhandled mmio` 错误后，Linux 内核终于能够顺利完成初始化并进入用户空间阶段。然而，在后续的启动过程中，系统卡在了 `[ OK ] Listening on Load/Save RF state /dev/rfkill watch`。这一行日志，表现为“卡死”现象。经过深入分析，判断这并非系统真正的崩溃或死锁，而是由于串口 `getty` 没有正常工作所导致的“伪卡死”。

在 `multi-user.target` 模式下，系统会默认尝试启动 `serial-getty@ttyAMA1.service`，以便提供串口终端登录功能。然而，如果 `/dev/ttyAMA1` 设备未被正确识别，或者对应的串口驱动未能正常加载，该服务便会进入反复重试状态，造成 `systemd` 启动流程在此处停滞，用户无法看到登录提示符，从而产生系统“卡住”的错觉(猜测是因为系统卡在了启动`serial-getty@ttyAMA1.service`之前的某个服务)。

考虑到当前环境对完整用户空间服务（如蓝牙、无线网络、音频等）的需求尚不迫切，我们选择通过在 `init=/bin/sh` 的方式，**绕过整个 `systemd` 初始化流程**，直接进入由 `BusyBox` 提供的 `Shell` 环境。这一方式不会加载任何服务、不会管理 `socket`、也不会尝试启动 `getty`，从而避免因串口控制台问题引发的阻塞，成功实现快速进入命令行界面的目标。最终，系统顺利进入了 `Shell` 交互环境，验证了内核及基础虚拟化功能的可用性，为后续进一步完善用户空间支持和设备驱动配置提供了稳定的调试基础。

```
chosen {
    stdout-path = "serial1:115200n8";
    bootargs = "clk_ignore_unused console=ttyAMA1,115200
earlycon=pl011,0x2800d000,115200 root=/dev/mmcblk0p1 rw rootwait init=/bin/sh";
};
```

3.1.8 virtio backend is too slow 问题分析

使用 `hvisor` 启动 Non-root Linux 时（使用 `virtio block`、`virtio serial`），日志持续打印如下警告和错误信息：

```
[WARN 2] (hvisor::device::virtio_trampoline:108) virtio backend is too slow,
please check it!
[ERROR 2] (hvisor::device::virtio_trampoline:112) virtio backend may have some
problem, please check it!
```

现象概述：

- **Root Linux** 运行正常，且 virtio 设备初始化成功；
- **Non-root Linux** 中 virtio 设备无法正常工作；
- 中断看似注入成功，但未生效；
- 当禁用 virtio 设备后，Non-root Linux 可以正常启动。

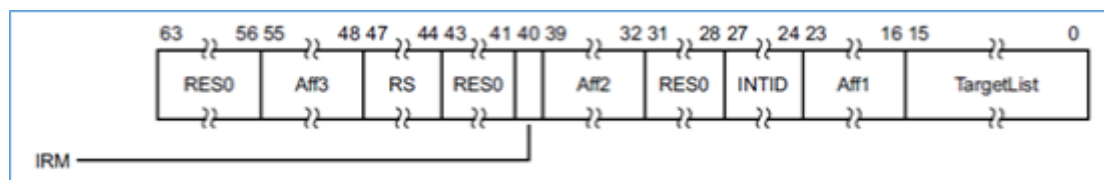
```
[DEBUG 2] (hvisor::device::virtio_trampoline:56) mmio virtio
handler,mmio.address: 0x0, mmio.size: 4, mmio.is_write: false, mmio.value:
0xffff80001168dc00,base: 0xa003c00
[DEBUG 2] (hvisor::device::virtio_trampoline:64) dev.base_address:
0x81bd9000, dev.is_enable: true
[DEBUG 2] (hvisor::device::virtio_trampoline:81) src_cpu: 0x2, address:
0xa003c00, size: 4, value: 0xffff80001168dc00, src_zone: 1, is_write: 0,
need_interrupt: 0
[DEBUG 2] (hvisor::device::virtio_trampoline:91) old cfg flag: 0x0,cpu2
[DEBUG 2] (hvisor::device::virtio_trampoline:210) push req to hvisor req
list, req_rear: 1, req_front: 0, req_list[1]: HvisorDeviceReq { src_cpu: 0,
address: 0, size: 0, value: 0, src_zone: 0, is_write: 0, need_interrupt: 0,
_padding: 0 }
[DEBUG 2] (hvisor::device::virtio_trampoline:96) need wakeup, sending ipi
to wake up virtio device
[DEBUG 2] (hvisor::arch::aarch64::ipi:63) send sgi to cpu_id: 0,
mpidr=0x200, sgi_num: 7
[DEBUG 2] (hvisor::arch::aarch64::ipi:74) write sgi sys value = 0x7020001
[INFO 3] (hvisor::arch::aarch64::trap:110) arch_handle_exit called 12901
times
[DEBUG 2] (hvisor::arch::aarch64::ipi:75) aff3 = 0x0, aff2 = 0x0, aff1 =
0x20000, irm = 0x0, sgi_id = 0x7000000, target_list = 0x1
[INFO 3] (hvisor::arch::aarch64::trap:111) Current exception reason: 3
[DEBUG 2] (hvisor::event:171) send_event: cpu_id: 0, ipi_int_id: 7,
event_id: 3
[WARN 2] (hvisor::device::virtio_trampoline:109) virtio backend is too
slow, please check it!
[ERROR 2] (hvisor::device::virtio_trampoline:113) virtio backend may have
some problem, please check it!
```

1.Virtio 请求正常触发，backend 中断未响应

通过日志追踪，virtio 前端设备触发了 MMIO 读请求，并设置了 `need_interrupt = true`，随后 hypervisor 内部发起了 IPI 中断：

```
[DEBUG 2] send sgi to cpu_id: 0, mpidr=0x200, sgi_num: 7
[DEBUG 2] (hvisor::arch::aarch64::ipi:74) write sgi sys value = 0x7020001
[DEBUG 2] aff3 = 0x0, aff2 = 0x0, aff1 = 0x20000, irm = 0x0, sgi_id = 0x7000000,
target_list = 0x1
```

`0x7020001` 对应 ICC_SGI1R_EL1 的值，表示 SGI 7 发往 Aff0 = 0、Aff1 = 2 (MPIDR = 0x200) 目标为 Root Linux 的 CPU (CPU0)，理论上这会唤醒 Root Linux 的 Virtio 后端处理线程。



2.SGI 中断未在 Root Linux 被处理.

查看 `gicv3_handle_irq_el1()` 函数发现只触发了虚拟定时器 (`irq_id==27`)，但从未触发 `irq_id == SGI_IPI_ID`。说明 SGI 虽然写入，但未能成功注入或处理：

```
if irq_id == SGI_IPI_ID {
    debug!("IPI received, irq_id = {}", irq_id);
    ipi_handled = check_events();
}
```

该分支始终未进入。

3.系统寄存器设置正确，GICC 接口正常初始化

系统中每个 CPU 都正确执行了 `gicc_init()` 和 `enable_ipi()`，其中 `DAIF` 中断屏蔽位已正确打开，且 `icc_igrpen1_el1 = 1`，`icc_pmr_el1 = 0xf0`，允许 Group 1 中断进入。Redistributor 的 `IGROUPR` 和 `ISENABLER` 也已设置。

4. 注入逻辑路径尚未打通

在当前实现中，虽然 `write_sysreg!(icc_sgirr_el1, val)` 成功发送 IPI，但实际 `vgic_inject_irq()` 并未将 SGI 设置进目标 CPU GICR 的 `SGIR` 寄存器，或未在 `vgic_pending_table` 中标记为 pending，导致 Root Linux 的 `pending_irq()` 无法枚举出 `irq_id == 7`，从而 `gicv3_handle_irq_el1()` 无法进入处理分支。

GIC Redistributor 地址冲突分析

通过对比发现，`hvisor` 在初始化 VGIC 时，为每个 CPU 分配的 Redistributor 地址如下：

```
[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu0 at 0x30880000
[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu1 at 0x308a0000
[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu2 at 0x308c0000
[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu3 at 0x308e0000
```

而 root Linux 和 non-root linux 启动日志显示，系统扫描 GICR_TYPER 并得到如下映射：

```
CPU0: found redistributor 200 region 0:0x00000000308c0000
CPU1: found redistributor 201 region 0:0x00000000308e0000
CPU2: found redistributor 0 region 0:0x0000000030880000
CPU3: found redistributor 100 region 0:0x00000000308a0000
```

原因分析：Linux 是根据设备树中 `/cpus` 节点的 `MPIDR` 值（即 `reg` 字段）确定 affinity，并据此扫描 GICR。

```
mpidr = (Aff3 << 32) | (Aff2 << 16) | (Aff1 << 8) | Aff0
```

```
0x00000000000000200 → Aff1 = 2, Aff0 = 0
0x00000000000000201 → Aff1 = 2, Aff0 = 1
0x00000000000000000 → Aff1 = 0, Aff0 = 0
0x00000000000000100 → Aff1 = 1, Aff0 = 0
```

节点名	reg (MPIDR)	Affinity (Aff1, Aff0)	说明
cpu@0	0x200	(2, 0)	Root Linux CPU0
cpu@1	0x201	(2, 1)	Root Linux CPU1
cpu@100	0x000	(0, 0)	Non-root Linux CPU2
cpu@101	0x100	(1, 0)	Non-root Linux CPU3

由于 Redistributor 地址应按 **MPIDR affinity (Aff1, Aff0)** 组合升序排列（而不是逻辑 CPU 编号），所以实际应为：.

```
gicr_base = 0x30880000
```

```
GICR for Aff1=0, Aff0=0 → 0x30880000 (non-root CPU2)
GICR for Aff1=1, Aff0=0 → 0x308a0000 (non-root CPU3)
GICR for Aff1=2, Aff0=0 → 0x308c0000 (root CPU0)
GICR for Aff1=2, Aff0=1 → 0x308e0000 (root CPU1)
```

hvisor 中原始分配逻辑存在问题：

```
let gicr_base = arch.gicr_base + cpu * PER_GICR_SIZE;
```

该方式仅按逻辑 CPU 编号依次排列，未考虑 MPIDR 与 affinity 的映射关系，导致 Linux 找到的 Redistributor 地址与 hvisor 注册不一致，**最终可能导致中断注入失败，从而影响 virtio 正常工作。**

修改为基于 MPIDR affinity 的 GICR 地址分配

为适配飞腾派平台，需重新实现GICR 分配逻辑：

```
pub fn host_gicr_base(id: usize) -> usize {
    if cfg!(feature = "mpidr_phytium") {
        /* phytium:
            GICR for Aff1=0, Aff0=0 → 0x30880000 (non-root CPU2)
            GICR for Aff1=1, Aff0=0 → 0x308a0000 (non-root CPU3)
            GICR for Aff1=2, Aff0=0 → 0x308c0000 (root CPU0)
            GICR for Aff1=2, Aff0=1 → 0x308e0000 (root CPU1)
        */
        static CPU_MPIDS: [usize; 4] = [0x200, 0x201, 0x000, 0x100]; // 逻辑
CPU0-3
        assert!(id < CPU_MPIDS.len());
        let mpidr = CPU_MPIDS[id];

        let aff0 = (mpidr >> 0) & 0xff;
        let aff1 = (mpidr >> 8) & 0xff;
        let cpu_interface_number = aff1 | aff0;
```

```

        GIC.get().unwrap().gicr_base + cpu_interface_number * PER_GICR_SIZE
    } else {
        assert!(id < consts::MAX_CPU_NUM);
        GIC.get().unwrap().gicr_base + id * PER_GICR_SIZE
    }
}

```

对应地，在 `vgicv3_mmio_init` 中替换为：

```

pub fn vgicv3_mmio_init(&mut self, arch: &HvArchZoneConfig) {
    if arch.gicd_base == 0 || arch.gicr_base == 0 {
        panic!("vgicv3_mmio_init: gicd_base or gicr_base is null");
    }

    self.mmio_region_register(arch.gicd_base, arch.gicd_size,
vgicv3_dist_handler, 0);
    self.mmio_region_register(arch.gits_base, arch.gits_size,
vgicv3_its_handler, 0);

    for cpu in 0..unsafe { consts::NCPU } {
        let gicr_base = if cfg!(feature = "mpidr_phytium") {
            host_gicr_base(cpu)
        } else {
            arch.gicr_base + cpu * PER_GICR_SIZE
        };
        info!("registering gicr cpu{} at {:#x?}", cpu, gicr_base);
        self.mmio_region_register(gicr_base, PER_GICR_SIZE,
vgicv3_redist_handler, cpu);
    }
}

```

报错原因源于 virtio 后端未及时响应，实为中断注入失败引发；深层根因是 GICR 分配逻辑未与 CPU 的 MPIDR 对齐。这样修改后即可保证 VGIC 注册的 GICR 区域与 Linux 启动过程中识别的一致，**确保 virtio 中断能够正确注入并被处理**。

3.1.9 GICR 映射重复 + LAST 标志设置错误

在解决了 GIC Redistributor 地址冲突问题后，虽然系统不再出现 `virtio backend is too slow` 的报错，但在 Linux 内核启动过程中仍然出现了卡死现象。具体表现为系统日志中输出如下内容后停滞不前：


```
[DEBUG 0] (hvisor::device::virtio_trampoline:116) non root receives value: 0x200000
[DEBUG 0] (hvisor::device::virtio_trampoline:116) non root receives value: 0x2000000000000000
[DEBUG 0] (hvisor::device::virtio_trampoline:116) non root receives value: 0x0
[ 2.714476] virtio_blk virtio0: [vda] 2097152 512-byte logical blocks (1.07 GB/1.00 GiB)
[ 2.725405] vda: detected capacity change from 0 to 1073741824
[DEBUG 0] (hvisor::device::virtio_trampoline:92) need wakeup, sending ipi to wake up virtio device
[DEBUG 0] (hvisor::arch::aarch64::ipi:63) send sgi to cpu_id: 2, mpidr=0x0, sgi_num: 7
[DEBUG 2] (hvisor::device::irqchip::gicv3:176) IPI received, irq_id = 7
[DEBUG 0] (hvisor::arch::aarch64::ipi:74) write sgi sys value = 0x7000001
[DEBUG 0] (hvisor::arch::aarch64::ipi:75) aff3 = 0x0, aff2 = 0x0, aff1 = 0x0, irm = 0x0, sgi_id = 0x7000000, target_list = 0x1
[DEBUG 0] (hvisor::event:171) send_event: cpu_id: 2, ipi_int_id: 7, event_id: 3
[DEBUG 0] (hvisor::device::virtio_trampoline:116) non root receives value: 0xb
[DEBUG 0] (hvisor::device::virtio_trampoline:92) need wakeup, sending ipi to wake up virtio device
[DEBUG 0] (hvisor::arch::aarch64::ipi:63) send sgi to cpu_id: 2, mpidr=0x0, sgi_num: 7
[DEBUG 2] (hvisor::device::irqchip::gicv3:176) IPI received, irq_id = 7
[DEBUG 0] (hvisor::arch::aarch64::ipi:74) write sgi sys value = 0x7000001
[DEBUG 0] (hvisor::arch::aarch64::ipi:75) aff3 = 0x0, aff2 = 0x0, aff1 = 0x0, irm = 0x0, sgi_id = 0x7000000, target_list = 0x1
[DEBUG 0] (hvisor::event:171) send_event: cpu_id: 2, ipi_int_id: 7, event_id: 3
[DEBUG 0] (hvisor::device::virtio_trampoline:92) need wakeup, sending ipi to wake up virtio device
[DEBUG 0] (hvisor::arch::aarch64::ipi:63) send sgi to cpu_id: 2, mpidr=0x0, sgi_num: 7
[DEBUG 2] (hvisor::device::irqchip::gicv3:176) IPI received, irq_id = 7
[DEBUG 0] (hvisor::arch::aarch64::ipi:74) write sgi sys value = 0x7000001
[DEBUG 0] (hvisor::arch::aarch64::ipi:75) aff3 = 0x0, aff2 = 0x0, aff1 = 0x0, irm = 0x0, sgi_id = 0x7000000, target_list = 0x1
[DEBUG 2] (hvisor::arch::aarch64::trap:285) HVC from CPU2, code:0x1, arg0:0x0, arg1:0x0
[DEBUG 0] (hvisor::event:171) send_event: cpu_id: 2, ipi_int_id: 7, event_id: 3
[DEBUG 2] (hvisor::hypercall:76) hypercall: code=HvVirtioInjectIrq, arg0=0x0, arg1=0x0
[DEBUG 2] (hvisor::arch::aarch64::ipi:63) send sgi to cpu_id: 0, mpidr=0x200, sgi_num: 7
[DEBUG 2] (hvisor::arch::aarch64::ipi:74) write sgi sys value = 0x7020001
[DEBUG 2] (hvisor::arch::aarch64::ipi:75) aff3 = 0x0, aff2 = 0x0, aff1 = 0x20000, irm = 0x0, sgi_id = 0x7000000, target_list = 0x1
[DEBUG 2] (hvisor::event:171) send_event: cpu_id: 0, ipi_int_id: 7, event_id: 2
[DEBUG 2] (hvisor::arch::aarch64::trap:299) HVC result = 0
```

1. GICR 地址映射重复

在 `board.rs` 中的 `ROOT_ZONE_MEMORY_REGIONS` 列表中，手动添加了 GICR 地址区域的映射：

```
pub const ROOT_ZONE_MEMORY_REGIONS: [HvConfigMemoryRegion; 11] = [
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x30880000, //gicr
        virtual_start: 0x30888000,
        size: 0x80000,
    },
];
```

而在 `vgicv3_mmio_init()` 函数中，又再次为每个 CPU 注册了 GICR 的 MMIO 区域：

```
self.mmio_region_register(gicr_base, PER_GICR_SIZE, vgicv3_redist_handler, cpu);
```

重复映射导致了内存访问冲突，进而影响到 Linux 内核对 GICR 的正常识别和初始化，故需要删除 `board.rs` 中对 GICR 区域的手动映射配置，让所有的 GICR 区域由 `vgicv3_mmio_init()` 函数统一管理。

2. GICR_TYPER.LAST 设置逻辑错误

删除后启动 Root linux 出现 `CPU0: mpidr 200 has no re-distributor!` 报错，没有找到 CPU0 对应的 GICR 基地址；在裸机 linux 启动日志和 hvisor 启动日志中可看到各个 CPU 对应的 gicr 地址如下，两者是一致的，理论上不应该会出现找不到 CPU0 对应的 GICR 基地址问题：

//裸机linux启动日志

CPU0: found redistributor 200 region 0:0x00000000308c0000

CPU1: found redistributor 201 region 0:0x00000000308e0000

CPU2: found redistributor 0 region 0:0x0000000030880000

CPU3: found redistributor 100 region 0:0x00000000308a0000

//hvisor启动日志

[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu0 at 0x30880000

[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu1 at 0x308a0000

[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu2 at 0x308c0000

[INFO 0] (hvisor::device::irqchip::gicv3::vgic:52) registering gicr cpu3 at 0x308e0000

查看linux源码分析Linux启动时如何扫描 GIC Redistributor 区域 (GICR) , 相关代码如下:

```
static int gic_iterate_rdist(int (*fn)(struct redist_region *, void __iomem *))
{
    int ret = -ENODEV;
    int i;
    // 遍历所有 Redistributor 区域
    for (i = 0; i < gic_data.nr_redist_regions; i++) {
        void __iomem *ptr = gic_data.redist_regions[i].redist_base;
        u64 typer;
        u32 reg;

        reg = readl_relaxed(ptr + GICR_PIDR2) & GIC_PIDR2_ARCH_MASK;
        if (reg != GIC_PIDR2_ARCH_GICv3 &&
            reg != GIC_PIDR2_ARCH_GICv4) { /* We're in trouble... */
            pr_warn("No redistributor present @%p\n", ptr);
            break;
        }
        // 遍历当前 Redistributor 区域中的所有 Redistributor 实例
        do {
            typer = gic_read_typer(ptr + GICR_TYPER);
            ret = fn(gic_data.redist_regions + i, ptr);
            if (!ret)
                return 0;

            if (gic_data.redist_regions[i].single_redist)
                break;

            if (gic_data.redist_stride) {
                ptr += gic_data.redist_stride;
            } else {
                ptr += SZ_64K * 2; /* Skip RD_base + SGI_base */
                if (typer & GICR_TYPER_VLPIS)
                    ptr += SZ_64K * 2; /* Skip VLPI_base + reserved page */
            }
        } while (!(typer & GICR_TYPER_LAST)); // GICR_TYPER_LAST == 1 (表示这是最后一个 Redistributor)
    }
}
```

```

return ret ? -ENODEV : 0;
}

```

Linux 内核在启动过程中会从 GICR 基地址开始依次扫描每个 CPU 的 Redistributor 区域，直到遇到设置了 `GICR_TYPER_LAST` 标志的寄存器页为止。结合 hvisor 启动日志，发现 Linux 目前只扫描到了 CPU2 (0x30880000) 和 CPU3 (0x308a0000) 的 Redistributor 区域，没有扫描到 CPU0 (0x308c0000) 和 CPU1 (0x308e0000) 的 Redistributor 区域：

```

[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:240) gicd-mmio: skip irq 284 access, reg = 0x68e0
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:240) gicd-mmio: skip irq 285 access, reg = 0x68e8
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:240) gicd-mmio: skip irq 286 access, reg = 0x68f0
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:240) gicd-mmio: skip irq 287 access, reg = 0x68f8
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:143) gicr(2) mmio = MMIOAccess {
  address: 0xffe8,
  size: 0x4,
  is_write: false,
  value: 0xffff80001170ffe8,
}
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:143) gicr(2) mmio = MMIOAccess {
  address: 0x8,
  size: 0x8,
  is_write: false,
  value: 0xffff800011700008,
}
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:143) gicr(2) mmio = MMIOAccess {
  address: 0x8,
  size: 0x8,
  is_write: false,
  value: 0xffff800011700008,
}
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:143) gicr(3) mmio = MMIOAccess {
  address: 0x8,
  size: 0x8,
  is_write: false,
  value: 0xffff800011720008,
}
[DEBUG 0] (hvisor::device::irqchip::gicv3::vgic:143) gicr(3) mmio = MMIOAccess {
  address: 0x8,
  size: 0x8,
  is_write: false,
  value: 0xffff800011720008,
}
}
0.000000] -----[ cut here ]-----
[ 0.000000] CPU0: mpidr 200 has no re-distributor!
[ 0.000000] WARNING: CPU: 0 PID: 0 at drivers/irqchip/irq-gic-v3.c:908 gic_init_bases+0x48c/0x590
[ 0.000000] Modules linked in:
[ 0.000000] CPU: 0 PID: 0 Comm: swapper/0 Not tainted 5.10.209-phytium-embedded-v2.2 #126
[ 0.000000] Hardware name: Phytium Pi Board (DT)
[ 0.000000] pstate: 60000085 (nZCv daIf -PAN -UA0 -TCO BTYPE=--)
[ 0.000000] pc : gic_init_bases+0x48c/0x590

```

在当前的 `vgicv3_redist_handler` 实现中，处理 `GICR_TYPER` 寄存器访问的逻辑如下：

```

match mmio.address {
  GICR_TYPER => {
    mmio_perform_access(gicr_base, mmio);
    if cpu == unsafe { consts::NCPU } - 1 { // NCPU=4
      mmio.value |= GICR_TYPER_LAST;
    }
  }
}

```

这段逻辑默认将最后一个 CPU（即 CPU3）设为最后一个 Redistributor，但实际上，在飞腾派平台上，**CPU0 的 GICR 区域才是最后一个 Redistributor**。因此，当 Linux 扫描到 CPU1 (0x308a0000) 时就提前遇到了 `LAST=1` 的标志，停止继续扫描，导致后续的 CPU0 (0x308c0000) 和 CPU1 (0x308e0000) 的 Redistributor 区域未被识别。

针对飞腾派平台的硬件特性，我们修改 `vgicv3_redist_handler` 中判断是否设置 `GICR_TYPER_LAST` 的逻辑：

```

match mmio.address {
  GICR_TYPER => {

```

```

        mmio_perform_access(gicr_base, mmio);
        if cfg!(feature = "mpidr_phytium"){
            if cpu == 1 { //cpu1 is the last cpu in phytium-pi
                mmio.value |= GICR_TYPER_LAST;
            }
        }else {
            if cpu == unsafe { consts::NCPU } - 1 {
                mmio.value |= GICR_TYPER_LAST;
            }
        }
    }
}
}

```

这样，Linux 内核在扫描时可以完整遍历所有 CPU 的 GICR 区域，正确识别每个 CPU 的 Redistributor，从而完成 GICv3 的初始化。修改完成后重新构建并运行 hvisor，Linux 内核成功完成了 GIC 初始化流程，能够识别所有 CPU 的 Redistributor 区域。串口输出完整，系统响应正常，Virtio 设备中断也能够正常注入。

3.1.10 翰博微e2000q教育开发板适配

飞腾派 CPU 是基于腾珑 E2000 系列的定制版本，而翰博威开发板采用标准的腾珑 E2000Q CPU。在完成飞腾派的 hvisor 适配后，我们着手适配翰博微E2000Q 开发板。初期适配工作举步维艰，主要原因是翰博微未开源其 U-Boot 和设备树，也缺乏相关技术文档。这使得我们无法直接获取适配所需的关键文件。为获取适配所需的**U-Boot 固件及设备树二进制文件**，我们采取了**逆向工程**手段：利用 **SP20B 编程器**，直接从翰博微开发板的 **Flash 存储芯片**中成功提取了这些关键文件，为 hvisor 在翰博微 E2000Q 开发板上的适配奠定了基础。**同时在适配过程中，也遇到了很多问题：**

1.启动hvisor时卡在starting kernel

在适配 hvisor 到翰博微 E2000Q 教育开发板时，我们遇到了系统在显示 "starting kernel" 后便**无任何输出且停止响应**的问题。初步排查时，我们曾怀疑是串口配置不正确。然而，考虑到我们已成功适配飞腾派，并且设备树中串口配置与飞腾派相同，因此很快排除了串口问题的可能性。

在此期间，我们一度怀疑通过 Flash 编程器逆向提取的设备树文件不完整，可能缺失关键节点。为进一步定位问题，我们尝试在 E2000Q 裸机环境下启动 Linux。在裸机启动过程中，我们意外发现，E2000Q 平台默认的**第一个启动 CPU 的 MPIDR 值为 0x00**。因为E2000Q 与飞腾派在处理器拓扑结构上一样，我们此前从未怀疑过启动顺序。然而，飞腾派上默认第一个启动的 CPU MPIDR 通常是 0x200（我们推测这是固件层面预设的），这与 hvisor 期望的第一个启动 CPU 之间产生了冲突，从而导致 hvisor 无法完成初始化，进而无法启动内核。为了解决**CPU MPIDR 冲突**问题，我们为 hvisor 添加了针对 E2000Q 特性的 `mpidr_e2000q` 支持。具体实现中，我们通过修改 `boot_cpuid_get` 函数来确保 hvisor 能够正确识别并处理 E2000Q 平台上的 CPU MPIDR 映射关系。

```

// 当启用 mpidr_e2000q 特性时编译此段代码
#[cfg(feature = "mpidr_e2000q")]
#[naked]
#[no_mangle]
pub unsafe extern "C" fn boot_cpuid_get() {
    core::arch::asm!(
        "
        mrs x17, mpidr_el1
        and x17, x17, #0xffff // low 16位

        cmp x17, #0x00 // cpu0
    )
}

```

```

        b.eq 3f
        cmp x17, #0x100          // cpu1
        b.eq 4f

        // 再检查其他核心
        cmp x17, #0x200          // cpu2
        b.eq 1f
        cmp x17, #0x201          // cpu3
        b.eq 2f

        mov x17, #-1             // unknown CPU
        b 5f

1:  mov x17, #2; b 5f             // back 2
2:  mov x17, #3; b 5f             // back 3
3:  mov x17, #0; b 5f             // back 0
4:  mov x17, #1                 // back 1
5:  ret
    ",
    options(noreturn)
);
}

```

2.多核启动时MPIDR 映射错误

在 `hvisor` 启动过程中，我们遇到了 "psci cpu on failed: InvalidParameters" 错误。这个错误明确指出，当前逻辑 CPU ID 未能正确映射到物理 CPU 的 MPIDR 值，导致通过 PSCI接口启动次核时失败。该问题源于不同 ARM 架构实现中，CPU 的逻辑 ID 与其物理 MPIDR 之间的映射关系可能存在差异。飞腾 E2000Q 平台有其特定的 MPIDR 编码规则，这与飞腾派等其他平台不同。为了确保 `hvisor` 能正确管理和启动 E2000Q 上的所有 CPU 核心，我们添加了如下针对 `mpidr_e2000q` 特性的适配：

- `cpu_start` 函数：逻辑 CPU ID 到物理 MPIDR 的映射

`cpu_start` 函数负责通过 PSCI 服务的 `cpu_on` 命令启动指定的 CPU 核心。为了正确调用 PSCI，需要将 `hvisor` 内部使用的逻辑 CPU ID 转换成目标物理 CPU 的实际 MPIDR 值。

```

pub fn cpu_start(cpuid: usize, start_addr: usize, opaque: usize) {
    let new_cpuid = match() {
        _ if cfg!(feature = "mpidr_phytium") => match cpuid {
            1 => 0x201,
            2 => 0x00,
            3 => 0x100,
            _ => panic!("Invalid cpuid: {}", cpuid),
        },
        _ if cfg!(feature = "mpidr_e2000q") => match cpuid {
            1 => 0x100,
            2 => 0x200,
            3 => 0x201,
            _ => panic!("Invalid cpuid: {}", cpuid),
        },
        _ => cpuid as u64 | 0x80000000,
    };
    psci::cpu_on(new_cpuid, start_addr as _, opaque as _).unwrap_or_else(|err| {
        println!("psci cpu_on failed: {:?}", err);
        if let psci::error::Error::AlreadyOn = err {

```

```

    } else {
        panic!("can't wake up cpu {}", cpuid);
    }
}
})
}

```

- `mpidr_to_cpuid` 函数: 物理 MPIDR 到逻辑 CPU ID 的逆向映射

该函数用于将从硬件寄存器（如 `MPIDR_EL1`）中读取到的物理 MPIDR 值转换为 `hvisor` 内部使用的逻辑 CPU ID，这用于识别当前运行的 CPU 是哪个逻辑核心。

```

pub fn mpidr_to_cpuid(mpidr: u64) -> u64 {
    #[cfg(feature = "mpidr_rockchip")]
    {
        (mpidr >> 8) & 0xff
    }

    #[cfg(feature = "mpidr_phytium")]
    {
        match (mpidr & 0xffff) as u32 {
            0x00 => 2,
            0x100 => 3,
            0x200 => 0,
            0x201 => 1,
            _ => panic!("Unknown MPIDR: {:#x}", mpidr),
        }
    }

    #[cfg(feature = "mpidr_e2000q")]
    {
        match (mpidr & 0xffff) as u32 {
            0x00 => 0,
            0x100 => 1,
            0x200 => 2,
            0x201 => 3,
            _ => panic!("Unknown MPIDR: {:#x}", mpidr),
        }
    }

    #[cfg(not(any(feature = "mpidr_rockchip", feature = "mpidr_phytium", feature =
"mpidr_e2000q" )))]
    {
        mpidr & 0xff00ffffff
    }
}

```

- `arch_send_event` 函数: 跨 CPU 核发送 SGI 中断

`arch_send_event` 函数用于向目标 CPU 发送软件生成中断（SGI）。在 GICv3 (Generic Interrupt Controller v3) 架构下，发送 SGI 需要构建一个包含目标 CPU 的 Affinity 级别和 SGI 编号的 `ICC_SGI1R_EL1` 寄存器值。由于在发送端无法直接访问目标 CPU 的完整 MPIDR 寄存器，我们通过逆向映射来从逻辑 `cpu_id` 推导出目标 Affinity 值。考虑到不同 CPU 实现（如 RK3568 和 RK3588）在 MPIDR 编码 Affinity 值上的差异，这里使用了条件编译来处理平台特定的 `cpu_id` 到中断目标 Affinity 的映射。

```

pub fn arch_send_event(cpu_id: u64, sgi_num: u64) {
    #[cfg(feature = "gicv3")]
    {
        /*Actually, the value passed to ICC_SGI1R_EL1 should be derived from
        the MPIDR of the target CPU. However, since we cannot access this
        register on the sender side, we have reverse-engineered a value
        here using the cpu_id.
        Due to differences in how some CPU implementations (e.g., RK3568 and
        RK3588)
        encode affinity values in MPIDR, we use conditional compilation to
        handle
        platform-specific mappings between cpu_id and interrupt target
        affinity.
        */
        let aff3: u64 = 0 << 48;
        let aff2: u64 = 0 << 32;
        let mut aff1: u64;
        let mut target_list: u64;
        let irm: u64 = 0 << 40;
        let sgi_id: u64 = sgi_num << 24;

        #[cfg(feature = "mpidr_rockchip")]
        {
            aff1 = cpu_id << 16;
            target_list = 1 << 0;
        }

        #[cfg(feature = "mpidr_phytium")]
        {
            let mpidr: u64 = match cpu_id {
                0 => 0x200,
                1 => 0x201,
                2 => 0x000,
                3 => 0x100,
                _ => panic!("Unsupported cpu_id: {}", cpu_id),
            };

            let aff0 = (mpidr >> 0) & 0xff;
            aff1 = ((mpidr >> 8) & 0xff) << 16;
            let aff2 = ((mpidr >> 16) & 0xff) << 32;
            let aff3 = ((mpidr >> 32) & 0xff) << 48;

            target_list = 1 << aff0;
            debug!("send sgi to cpu_id: {}, mpidr=0x{:x}, sgi_num: {}",
                cpu_id, mpidr, sgi_num);
        }

        #[cfg(feature = "mpidr_e2000q")]
        {
            let mpidr: u64 = match cpu_id {
                0 => 0x00,
                1 => 0x100,
                2 => 0x200,
                3 => 0x201,
                _ => panic!("Unsupported cpu_id: {}", cpu_id),
            };

```

```

};

let aff0 = (mpidr >> 0) & 0xff;
aff1 = ((mpidr >> 8) & 0xff) << 16;
let aff2 = ((mpidr >> 16) & 0xff) << 32;
let aff3 = ((mpidr >> 32) & 0xff) << 48;

target_list = 1 << aff0;
debug!("send sgi to cpu_id: {}, mpidr=0x{:x}, sgi_num: {}",
cpu_id, mpidr, sgi_num);
}
#[cfg(not(any(
    feature = "mpidr_rockchip",
    feature = "mpidr_phytium",
    feature = "mpidr_e2000q"
)))]
{
    aff1 = 0 << 16;
    target_list = 1 << cpu_id;
}

let val: u64 = aff3 | aff2 | aff1 | irm | sgi_id | target_list;
write_sysreg!(icc_sgi1r_el1, val);
debug!("write sgi sys value = {:#x}", val);
}
#[cfg(feature = "gicv2")]
{
    let sgi_id: u64 = sgi_num;
    let target_list: u64 = 1 << cpu_id;
    set_sgi_irq(sgi_id as usize, target_list as usize, 0);
}
}

```

3.解决内存节点定义错误导致的 Linux 启动失败

```

[WARN 0] (hvisor::memory::mmio:96) Zone 0 unhandled mmio fault MMIOAccess {
    address: 0x27ffff000,
    size: 0x1,
    is_write: true,
    value: 0xfffffdfffe437000,
}
[ERROR 0] (hvisor::arch::aarch64::trap:243) mmio_handle_access:
[src/memory/mmio.rs:97:13] Invalid argument
[ERROR 0] (hvisor::panic:24) panic occurred: PanicInfo {
    payload: Any { .. },
    message: Some(
        root zone has some error,
    ),
    location: Location {
        file: "src/zone.rs",
        line: 292,
        col: 9,
    },
    can_unwind: true,
    force_no_backtrace: false,
}

```

```
}
```

这些错误信息 `unhandled mmio fault` 和 `Invalid argument`，表示 `hvisor` 在处理内存映射输入/输出 (MMIO) 访问时遇到了问题，最终在内存区域管理 (`src/zone.rs`) 中触发了 `panic`。这通常是由于操作系统或管理程序尝试访问未映射或权限不正确的内存地址所致。经过深入排查，我们发现问题出在设备树中 `memory` 节点的定义。通过 `dtc` 工具将逆向获取的 `.dtb` 文件导出为 `.dts` 文本格式后，我们观察到原始 `memory` 节点的定义如下所示：

```
//memory@00 {  
//  device_type = "memory";  
//  reg = <0x00 0x80000000 0x02 0x00>;  
//};
```

该 `reg` 属性定义了一个从物理地址 `0x00` 开始、大小为 `0x2_0000_0000` 字节的内存区域，这等同于 **8GB** 内存。然而，实际的翰博威 E2000Q 教育开发板**仅配备 4GB 内存**。这种不正确的内存范围定义导致 Linux 内核在启动时尝试访问超出实际物理内存边界的地址，进而触发了 `hvisor` 的 MMIO 错误和随后的 `panic`。

为了解决此问题，我们将设备树中 `memory` 节点的 `reg` 属性修改为与开发板实际内存容量相匹配的值：

```
memory@80000000 {  
    device_type = "memory";  
    reg = <0x0 0x80000000 0x00 0x80000000>;  
};
```

4.加载hvisor.ko内核失败

在成功启动 Root Linux 后，我们尝试加载 `hvisor.ko` 内核模块以启动 Non-Root Linux，但遇到了如下错误：

```
root@(none):/home# insmod hvisor.ko  
[ 12.524631] hvisor: loading out-of-tree module taints kernel.  
[ 12.531169] irq: type mismatch, failed to map hwirq-64 for interrupt-  
controller@30800000!  
[ 12.539430] hvisor cannot register IRQ, err is -22  
insmod: ERROR: could not insert module hvisor.ko: Invalid parameters
```

错误日志显示 `irq: type mismatch, failed to map hwirq-64` 和 `hvisor cannot register IRQ, err is -22`，明确指出 `hvisor.ko` 在注册中断时因中断类型不匹配而失败，导致模块无法加载（错误码 -22，即 `EINVAL`，参数无效）。

冲突的设备树节点定义如下：


```
usb2@31800000 {
    compatible = "phytium,e2000-usb2";
    reg = <0x00 0x31800000 0x00 0x80000 0x00 0x31990000 0x00 0x10000>;
    interrupts = <0x00 0x20 0x04>; // <-- 此处中断号为 0x20
    status = "ok";
    dr_mode = "host";
};

hvisor_virtio_device {
    compatible = "hvisor";
    interrupt-parent = <0x01>;
    interrupts = <0x00 0x20 0x01>; // <-- 此处中断号也为 0x20
};
```

从上述配置中可以看到，usb2 设备和 hvisor_virtio_device 都尝试使用中断号 0x20。在 GIC 体系中，中断号是唯一的资源标识符。当两个不同的设备请求注册同一个中断号时，就会导致冲突并阻止模块的正常加载。由于当前阶段尚无需使用 USB2 设备，我们采取的临时解决方案是禁用设备树中的 usb2@31800000 节点，以解除中断号冲突。通过将 usb2 节点的状态 status 属性设置为 "disabled"，可以避免其在内核启动时注册该中断。完成此修改后，hvisor.ko 模块得以成功加载，未来若需启用 USB2 功能，则需重新为 hvisor_virtio_device 分配设备中断号。

5.Root Linux 网络服务适配

在为 Root Linux 配置网络时，我们尝试使用 `ifconfig eth0 192.168.137.2` 命令，但系统返回了 "SIOCSIFADDR: No such device" 和 "eth0: ERROR while getting interface flags: No such device" 的错误。这表明以太网接口 eth0 未被内核成功识别和初始化。我们首先检查了设备树中网络设备的定义：

```
ethernet@32012000 {
    compatible = "phytium,gem";
    reg = <0x00 0x32012000 0x00 0x2000>;
    interrupts = <0x00 0x44 0x04 0x00 0x45 0x04 0x00 0x46 0x04 0x00 0x47
0x04>;

    clock-names = "pclk\0hclk\0tx_clk\0tsu_clk";
    clocks = <0x13 0x14 0x14 0x13>;
    magic-packet;
    status = "okay";
    phy-mode = "sgmii";
    use-mii;
};
```

尽管设备树中明确定义了网络设备 ethernet@32012000 且 status 属性为 "okay"，但内核仍未能识别它。这使我们怀疑当前的 Linux 内核可能缺少针对 "phytium,gem" 这个 compatible 字符串的相应驱动程序。为了验证这一猜测并寻找正确的驱动匹配，我们查阅了飞腾派（该平台网络直通功能正常）的网络设备节点定义。我们发现飞腾派使用的 compatible 字段内容为 "cdns,phytium-gem-1.0"。由于飞腾派的网络功能正常，推测 Linux 内核中很可能已经包含了这个驱动。基于此分析，我们将瀚博威 E2000Q 设备树中网络设备的 compatible 字段从 "phytium,gem" 修改为 "cdns,phytium-gem-1.0"：

```

ethernet@32012000 {
    // compatible = "phytium,gem"; // 旧的兼容性字符串
    compatible = "cdns,phytium-gem-1.0"; // 修改为与现有内核驱动匹配的兼容性字符串

    reg = <0x00 0x32012000 0x00 0x2000>;
    interrupts = <0x00 0x44 0x04 0x00 0x45 0x04 0x00 0x46 0x04 0x00 0x47
0x04>;

    clock-names = "pc1k\0hc1k\0tx_clk\0tsu_clk";
    clocks = <0x13 0x14 0x14 0x13>;
    magic-packet;
    status = "okay";
    phy-mode = "sgmii";
    use-mii;
};

```

重新启动 Root Linux 后，以太网接口 `eth0` 被内核成功识别和初始化，网络服务得以正常配置和使用。这证实了问题确实是由设备树中的 `compatible` 字符串与内核驱动不匹配所导致。

3.2 启动hvisor

在飞腾派平台上，`hvisor` 可以通过 U-Boot 手动加载并启动。流程包括：设置 IP 地址、加载镜像与设备树文件、执行启动命令。镜像可以从 SD 卡加载，也可以通过 TFTP 网络方式传输：

启动流程（基于 TFTP）

1.设置环境变量

<code>setenv serverip 192.168.137.1</code>	<code># TFTP 服务器地址</code>
<code>setenv ipaddr 192.168.137.2</code>	<code># 开发板自身 IP</code>
<code>setenv loadaddr 0x90100000</code>	<code># 加载 hvisor.bin 的地址</code>
<code>setenv fdt_addr 0x90000000</code>	<code># 加载 hvisor 设备树的地址</code>
<code>setenv zone0_kernel_addr 0xa0400000</code>	<code># Zone0 内核镜像加载地址</code>
<code>setenv zone0_fdt_addr 0xa0000000</code>	<code># Zone0 内核设备树加载地址</code>

2.通过 TFTP 加载镜像与设备树

```

tftp ${loadaddr} ${serverip}:hvisor.bin
tftp ${fdt_addr} ${serverip}:phytium-pi-board-v2.dtb
tftp ${zone0_kernel_addr} ${serverip}:Image
tftp ${zone0_fdt_addr} ${serverip}:linux1.dtb

```



```

TFTP from server 192.168.137.1; our IP address is 192.168.137.2
Filename 'linux1.dtb'.
Load address: 0xa0000000
Loading: #####
4.9 KiB/s
done
Bytes transferred = 28022 (6d76 hex)
## Booting kernel from Legacy Image at 90100000 ...
Image Name: hvisor_img
Image Type: AArch64 Linux Kernel Image (uncompressed)
Data Size: 692304 Bytes = 676.1 KiB
Load Address: 90100000
Entry Point: 90100000
Verifying Checksum ... OK
## Flattened Device Tree blob at 90000000
Booting using the fdt blob at 0x90000000
Loading Kernel Image
Loading Device Tree to 00000000f9c27000, end 00000000f9c31a5d ... OK
run in ft_board_setup
fdt_addr 00000000f9c27000
mb_count = 0x2
mb_blocks[0].mb_size = 0x7c000000
mb_blocks[1].mb_size = 0x80000000
fdt : can not find /memory@01 node
fdt : add node memory@01
fdt : dram size 0x100000000 update successfully

Starting kernel ...

Hello, start HVISOR at 0x90100000!
Heap allocator initialization finished: 0x901ac010..0x902ac010
heap_test passed!
Booting CPU 0: 0x902cf000 arch:0x902cf018, DTB: 0xf9c27000
Using 4 cpu(s) on this system.
I/TC: Secondary CPU 1 initializing
I/TC: Secondary CPU 1 switching to normal world boot
Hello, start HVISOR at 0x90100000!
Booting CPU 1: 0x9034f000 arch:0x9034f018, DTB: 0xf9c27000
I/TC: Secondary CPU 2 initializing
I/TC: Secondary CPU 2 switching to normal world boot
Hello, start HVISOR at 0x90100000!
Booting CPU 2: 0x90I3/cTfC0:0 0Se caorncdha:ry 0CxP9U0 33c fi0n1i8ti,a lDiTzBi:n g
0xf9c27000
I/TC: Secondary CPU 3 switching to normal world boot
Hello, start HVISOR at 0x90100000!
Booting CPU 3: 0x9044f000 arch:0x9044f018, DTB: 0xf9c27000
Primary CPU 0 has entered.
Secondary CPU 1 has entered.
Secondary CPU 3 has entered.
Secondary CPU 2 has entered.

```

注意事项

- `bootm` 启动的是一个裸机程序 (hvisor) , 因此 `initrd` 参数留空;
- `zone0_kernel_addr` 与 `zone0_fdt_addr` 通常会在 hvisor 的配置文件中被解析使用;
- 若使用 SD 卡方式加载镜像, `tftp` 命令可替换为 `fatload mmc` 命令, 路径格式为 `fatload mmc 0:1 ${addr} /path/to/file`;
- 请确保 TFTP 服务已开启, 镜像文件已放置于 TFTP 根目录, 且 U-Boot 配置中启用了网络功能。

3.3 启动zone0(root linux)

在 `hvisor` 中, Zone0 作为 Root Linux, 承担基础 I/O 管理和系统引导的职责。为了实现其正常启动, 需同时配置设备树 (`zone0.dts`) 和 `board.rs` 中的硬件映射与中断资源。本节将详细说明各部分配置。

3.3.1 zone0设备树配置 (zone0.dts)

1.添加hvisor_virtio_device设备树节点

Zone0 中需要声明虚拟设备接口，使其能够与 Hypervisor 管理的 virtio 后端通信。通过如下设备树节点声明：

```
hvisor_virtio_device {
    compatible = "hvisor";
    interrupt-parent = <0x01>;
    interrupts = <0x00 0x20 0x01>;
};
```

该节点指明了 virtio 管理设备的中断来源，便于内核注册中断处理器。

2.配置内存与保留区域

Zone0 的内存区域从 0x80000000 起始，大小为 2GB。同时，为防止 Zone0 访问 hvisor 本身及其他 Zone 所使用的内存区域，在设备树中通过 reserved-memory 节点标记这些区域为 no-map：

```
memory@80000000 {
    device_type = "memory";
    reg = <0x0 0x80000000 0x00 0x80000000>;
};
reserved-memory {
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    ranges;

    hvisor@90100000 {
        no-map;
        reg = <0x00 0x90100000 0x00 0x400000>;
    };
    dtbfile@90000000 {
        no-map;
        reg = <0x00 0x90000000 0x00 0x080000>;
    };
    nonroot@b0000000 {
        no-map;
        reg = <0x00 0xb0000000 0x00 0x30000000>;
    };
};
```

3.设置内核启动参数

通过 chosen 节点配置 Linux 启动参数：

```
chosen {
    stdout-path = "serial1:115200n8";
    bootargs = "clk_ignore_unused console=ttyAMA1,115200
earlycon=pl011,0x2800d000,115200 root=/dev/mmcblk0p1 rw rootwait init=/bin/sh";
};
```

3.3.2 zone0配置 (board.dts)

Zone0 的配置通过 `board.rs` 中的若干常量完成, 包括内核加载地址、CPU分配、中断映射及内存映射等。

`board.rs`:zone0启动的配置文件,根据zone0设备树所设定的设备节点, 在board.rs上分配相同的地址空间给zone0:

1. 基本参数

```
use crate::{arh::zone::HvArchZoneConfig, config::*};

pub const BOARD_NAME: &str = "phytium-pi";

pub const BOARD_NCPUS: usize = 4;
// zone0 设备树文件加载地址
pub const ROOT_ZONE_DTB_ADDR: u64 = 0xa0000000;
// zone0 内核加载地址
pub const ROOT_ZONE_KERNEL_ADDR: u64 = 0xa0400000;
// zone0 内核启动地址
pub const ROOT_ZONE_ENTRY: u64 = 0xa0400000;
// zone0启动时所要使用的CPU
pub const ROOT_ZONE_CPUS: u64 = (1 << 1) | (1 << 0);

pub const ROOT_ZONE_NAME: &str = "root-linux";
```

2. 内存映射

Zone0 需拥有自身运行所需的 RAM 与各类外设 (USB、网卡、GIC、SRAM、Mailbox 等) 访问权限。以下结构体数组声明了这些物理区域的映射关系:

```
pub const ROOT_ZONE_MEMORY_REGIONS: [HvConfigMemoryRegion; 12] = [
    // ram
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_RAM,
        physical_start: 0x80000000,
        virtual_start: 0x80000000,
        size: 0x80000000,
    },
    // soc@0
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x28000000,
        virtual_start: 0x28000000,
        size: 0x00100000,
    },
    //iommu
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x30000000,
        virtual_start: 0x30000000,
        size: 0x800000,
    },
    // GIC
    HvConfigMemoryRegion {
```

```

        mem_type: MEM_TYPE_IO,
        physical_start: 0x30800000,
        virtual_start: 0x30800000,
        size: 0x00100000,
    },
    // ethernet0
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x3200c000,
        virtual_start: 0x3200c000,
        size: 0x00002000,          // 8KB
    },
    // USB
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x31800000, // usb2@31800000
        virtual_start: 0x31800000,
        size: 0x00080000,          // 512KB
    },
    // USB2 @32800000 - Host Mode
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x32800000,
        virtual_start: 0x32800000,
        size: 0x00040000, // 256KB
    },
    // USB2 @32840000 - Host Mode
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x32840000,
        virtual_start: 0x32840000,
        size: 0x00040000, // 256KB
    },
    // USB3 @31a08000 - XHCI Controller
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x31a08000,
        virtual_start: 0x31a08000,
        size: 0x00018000, // 96KB
    },
    // USB3 @31a28000 - XHCI Controller
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x31a28000,
        virtual_start: 0x31a28000,
        size: 0x00018000, // 96KB
    },
    // mailbox
    HvConfigMemoryRegion {
        mem_type: MEM_TYPE_IO,
        physical_start: 0x32a00000,
        virtual_start: 0x32a00000,
        size: 0x1000,
    },

```

```
//sram
HvConfigMemoryRegion {
    mem_type: MEM_TYPE_IO,
    physical_start: 0x32a10000,
    virtual_start: 0x32a10000,
    size: 0x2000,
}
];
```

3. 中断配置

为了正确注入中断，必须显式列出所有在设备树中声明的中断号：

```
pub const ROOT_ZONE_IRQS: [u32; 13] = [46, 54, 64, 75, 76, 78, 87, 104, 105, 116,
133, 138, 191];
```

4. 中断控制器结构体

在 `HvArchZoneConfig` 结构体中配置 GICv3 的 GICD、GICR 区域，使 Hypervisor 能正确代理与注入中断：

```
pub const ROOT_ARCH_ZONE_CONFIG: HvArchZoneConfig = HvArchZoneConfig {
    gicd_base: 0x30800000,
    gicd_size: 0x20000,
    gicr_base: 0x30880000,
    gicr_size: 0x80000,
    gicc_base: 0x0,
    gicc_size: 0x0,
    gicc_offset: 0x0,
    gich_base: 0x0,
    gich_size: 0x0,
    gicv_base: 0x0,
    gicv_size: 0x0,
    gits_base: 0x0,
    gits_size: 0x0,
};
```

此配置中 `gicc/gich/gicv/gits` 可暂时置零，仅启用 GICD 与 GICR 即可满足基础中断转发功能。


```
[INFO 0] (hvisor::zone:265) zone cpu set: 0b11
[INFO 0] (hvisor:166) CPU 0 hv_pt_install OK.
[INFO 1] (hvisor:166) CPU 1 hv_pt_install OK.
[WARN 3] (hvisor:161) zone is not created for cpu 3
[INFO 0] (hvisor::device::irqchip::gicv3:122) PMR set to 0x80 for CPU0
[INFO 1] (hvisor::device::irqchip::gicv3:122) PMR set to 0x80 for CPU1
[INFO 0] (hvisor::device::irqchip::gicv3:123) ICC_CTLR_EL1 = 0x402
[INFO 3] (hvisor:166) CPU 3 hv_pt_install OK.
[INFO 0] (hvisor::device::irqchip::gicv3:124) ICC_PMR_EL1 = 0xf0
[INFO 1] (hvisor::device::irqchip::gicv3:123) ICC_CTLR_EL1 = 0x402
[INFO 0] (hvisor::device::irqchip::gicv3:125) ICC_IGRPEN1_EL1 = 0x1
[INFO 1] (hvisor::device::irqchip::gicv3:124) ICC_PMR_EL1 = 0xf0
[INFO 0] (hvisor::device::irqchip::gicv3:132) gicc init done, sdei_ver = 281474976710656
[INFO 1] (hvisor::device::irqchip::gicv3:125) ICC_IGRPEN1_EL1 = 0x1
[INFO 0] (hvisor::device::irqchip::gicv3:133) cpu:0,gicc init done, sdei_ver = 281474976710656
[INFO 1] (hvisor::device::irqchip::gicv3:132) gicc init done, sdei_ver = 281474976710656
[INFO 0] (hvisor::device::irqchip::gicv3::gicr:63) CPU0 GICR PENDBASER = 0x43b0000000
[INFO 1] (hvisor::device::irqchip::gicv3:133) cpu:1,gicc init done, sdei_ver = 281474976710656
[INFO 3] (hvisor::device::irqchip::gicv3:122) PMR set to 0x80 for CPU3
[INFO 1] (hvisor::device::irqchip::gicv3::gicr:63) CPU0 GICR PENDBASER = 0x43b0000000
[INFO 3] (hvisor::device::irqchip::gicv3:123) ICC_CTLR_EL1 = 0x402
[WARN 2] (hvisor:161) zone is not created for cpu 2
[INFO 3] (hvisor::device::irqchip::gicv3:124) ICC_PMR_EL1 = 0xf0
[INFO 2] (hvisor:166) CPU 2 hv_pt_install OK.
[INFO 3] (hvisor::device::irqchip::gicv3:125) ICC_IGRPEN1_EL1 = 0x1
[INFO 2] (hvisor::device::irqchip::gicv3:122) PMR set to 0x80 for CPU2
[INFO 3] (hvisor::device::irqchip::gicv3:132) gicc init done, sdei_ver = 281474976710656
[INFO 2] (hvisor::device::irqchip::gicv3:123) ICC_CTLR_EL1 = 0x402
[INFO 3] (hvisor::device::irqchip::gicv3:133) cpu:3,gicc init done, sdei_ver = 281474976710656
[INFO 2] (hvisor::device::irqchip::gicv3:124) ICC_PMR_EL1 = 0xf0
[INFO 3] (hvisor::device::irqchip::gicv3::gicr:63) CPU0 GICR PENDBASER = 0x43b0000000
[INFO 2] (hvisor::device::irqchip::gicv3:125) ICC_IGRPEN1_EL1 = 0x1
[INFO 2] (hvisor::device::irqchip::gicv3:132) gicc init done, sdei_ver = 281474976710656
[INFO 2] (hvisor::device::irqchip::gicv3:133) cpu:2,gicc init done, sdei_ver = 281474976710656
[INFO 2] (hvisor::device::irqchip::gicv3::gicr:63) CPU0 GICR PENDBASER = 0x43b0000000
[INFO 0] (hvisor:153) Primary CPU init late...
[INFO 2] (hvisor::percpu:70) CPU2: Idling the CPU before starting VM...
[INFO 0] (hvisor::percpu:73) CPU0: Running the VM...
[INFO 2] (hvisor::memory:mm:112) region.start: 0x0
[INFO 0] (hvisor::arch::aarch64::cpu:195) cpu 0 started at 0xa0400000
[INFO 1] (hvisor::percpu:70) CPU1: Idling the CPU before starting VM...
[INFO 2] (hvisor::arch::aarch64::cpu:233) c pu 2 start parking
37m[INFO 1] (hvisor::arch::aarch64::cpu:233) cpu 1 start parking
[INFO 3] (hvisor::percpu:70) CPU3: Idling the CPU before starting VM...
m0.[0INFO [m 030] [37m(hvisor::arch::aarch64::cpu:2)33) [32mcpup 30 start parking
ting Linux on physical CPU 0x0000000200f0x700f30341
[ 0.000000] Linux version 5.10.209-phytium-embedded-v2.2 (b@PC-20241012EPJE) (aarch64-none-linux-gnu-gcc (GNU Toolchain for
10.2-2020.11 (arm-10.16)) 2.35.1.20201028) #126 SMP PREEMPT Fri Jun 27 09:35:39 CST 2025
[ 0.000000] Machine model: Phytium Pi Board
[ 0.000000] earlycon: pl11 at MMIO 0x000000002800d000 (options '115200')
[ 0.000000] printk: bootconsole [pl11] enabled
[ 0.000000] efi: UEFI not found.
[ 0.000000] Zone ranges:
[ 0.000000] DMA [mem 0x0000000080000000-0x00000000ffffffff]
[ 0.000000] DMA32 empty
[ 0.000000] Normal empty
[ 0.000000] Movable zone start for each node
[ 0.000000] Early memory node ranges
[ 0.000000] node 0: [mem 0x0000000080000000-0x000000008fffffffff]
[ 0.000000] node 0: [mem 0x0000000090000000-0x000000009007ffff]
[ 0.000000] node 0: [mem 0x0000000090080000-0x00000000900fffff]
[ 0.000000] node 0: [mem 0x0000000090100000-0x00000000904fffff]
[ 0.000000] node 0: [mem 0x0000000090500000-0x0000000090affffff]
```

```

[ 1.430462] mmc0: error -22 whilst initialising SDIO card
[ 1.436111] phytiwm-mci-platform 28001000.mmc: phytiwm_mci_probe 114: probe phytiwm mci successful.
[ 1.445926] ledtrig-cpu: registered to indicate activity on CPUs
[ 1.452474] usbcore: registered new interface driver usbhid
[ 1.458188] usbhid: USB HID core driver
[ 1.462682] phytiwm-mbox 32a00000.mailbox: Phytium SoC Mailbox registered
[ 1.470421] usbcore: registered new interface driver snd-usb-audio
[ 1.477004] phytiwm-mci-platform 28001000.mmc: card claims to support voltages below defined range
[ 1.486683] genirq: irq_chip 28036000.gpio did not update eff. affinity mask of irq 100
[ 1.496188] NET: Registered protocol family 17
[ 1.500811] 9pnet: Installing 9P2000 support
[ 1.505356] Key type dns_resolver registered
[ 1.510050] registered taskstats version 1
[ 1.514199] Loading compiled-in X.509 certificates
[ 1.519810] arm-scmi firmware:scmi: SCMI Notifications - Core Enabled.
[ 1.526407] mmc0: host does not support reading read-only switch, assuming write-enable
[ 1.534476] mmc0: new high speed SDXC card at address aaaa
[ 1.534686] arm-scmi firmware:scmi: SCMI Protocol v2.0 'Phytium:E2000' Firmware version 0x2050000
[ 1.540410] mmcblk0: mmc0:aaaa SD64G 59.5 GiB
[ 1.554067] mmc1: new high speed SDIO card at address 0001
[ 1.559964] mmcblk0: p1
[ 1.673269] cpu cpu0: EM: created perf domain
[ 1.678594] of_cfs_init
[ 1.681089] of_cfs_init: OK
[ 1.683980] clk: Not disabling unused clocks
[ 1.688274] ALSA device list:
[ 1.691242] #0: phytiwm,pe220x-i2s-audio
[ 1.695454] uart-pl011 2800d000.uart: no DMA platform data
[ 1.730595] EXT4-fs (mmcblk0p1): recovery complete
[ 1.736533] EXT4-fs (mmcblk0p1): mounted filesystem with ordered data mode. Opts: (null)
[ 1.744691] VFS: Mounted root (ext4 filesystem) on device 179:1.
[ 1.757894] devtmpfs: mounted
[ 1.761541] Freeing unused kernel memory: 2432K
[ 1.766141] Run /bin/sh as init process
/bin/sh: 0: can't access tty; job control turned off
# bash
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@(none):/#
root@(none):/# mount -t proc proc /proc
root@(none):/# mount -t sysfs sysfs /sys
root@(none):/# cat /proc/cpuinfo
processor       : 0
BogoMIPS      : 100.00
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 cpuid sha3 sm3 sm4 sha512
CPU implementer : 0x70
CPU architecture: 8
CPU variant    : 0x0
CPU part       : 0x303
CPU revision   : 4

processor       : 1
BogoMIPS      : 100.00
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 cpuid sha3 sm3 sm4 sha512
CPU implementer : 0x70
CPU architecture: 8
CPU variant    : 0x0
CPU part       : 0x303
CPU revision   : 4

root@(none):/# ls
bin      core      enable  lib      mnt      polarity  run  srv  usr
boot     dev       etc     lost+found  opt      proc      sbin  sys  var
brightness  duty_cycle  home    media    period   root      snap  tmp
root@(none):/#

```

进入bash

成功识别到CPU0和CPU1

3.4 启动zone1(non-root linux)

在本项目中，hvisor 作为 Type-1 Hypervisor，运行于 EL2 特权级，负责提供完整的虚拟化支持。系统中通过运行在 Root Zone（即 zone0）的 Linux 来管理其他虚拟机（Zone），其中包括对 zone1 的启动和配置管理。

3.4.1 管理工具 hvisor-tool 简介

虚拟机的创建、销毁、Virtio 设备启停等操作由 hvisor-tool 工具链负责，该工具链由命令行工具、Virtio 守护进程和内核模块三部分组成：

- **命令行工具**：负责解析用户命令并与内核模块通信；
- **Virtio 守护进程**：在用户态运行，负责模拟 virtio-blk、virtio-console 等设备；
- **内核模块（hvisor.ko）**：运行于 zone0 内核空间，作为用户态工具与 Hypervisor 的桥梁。

hvisor-tool 开源仓库地址为：<https://github.com/syswonder/hvisor-tool>。

工具编译命令如下：

```
make all ARCH=<arch> LOG=<log> KDIR=/path/to/your-linux
```

完成后可在 `tools/hvisor/` 和 `driver/hvisor.ko` 中分别获得命令行工具和内核模块，将其部署到 Root Linux 的文件系统(SD卡)中，即可使用。

3.4.2 精简Zone1 设备树 (linux2.dts)

在启动 `zone1` 前，需要为其提供一份精简版的设备树 (`linux2.dts`)，该设备树应仅保留 `zone1` 所需的基本硬件资源，例如串口、Virtio MMIO、定时器等。重要原则是**避免与 Root Zone 占用资源冲突**，即 `zone0` 和 `zone1` 不得访问重叠的内存地址空间、中断号或设备寄存器区域。

设备树中需配置如下关键项：

- **CPU 节点**：配置由 `zone1` 使用的逻辑核心 (CPU2 和 CPU3) ；
- **内存区域**：为 `zone1` 分配独立的 RAM 空间，例如 `0xb0000000` 开始的 768MB；
- **中断控制器**：配置 GICv3 的中断寄存器地址；
- **Virtio 设备节点**：配置 `virtio-mmio` 设备，如块设备 (blk) 和控制台 (console) ；
- **chosen 节点**：配置启动参数、控制台输出路径。

精简后的zone1设备树linux2.dts内容如下：

```
/dts-v1/;

/memreserve/    0x0000000080000000 0x0000000000010000;
/ {
    compatible = "phytium,pe2204";
    interrupt-parent = <0x01>;
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    model = "Phytium Pi Board";

    aliases {
        serial1 = "/soc/uart@2800d000";
    };

    psci {
        compatible = "arm,psci-1.0";
        method = "smc";
        cpu_suspend = <0xc4000001>;
        cpu_off = <0x84000002>;
        cpu_on = <0xc4000003>;
        sys_poweroff = <0x84000008>;
        sys_reset = <0x84000009>;
    };

    firmware {

        scmi {
            compatible = "arm,scmi";
            mboxs = <0x02 0x00>;
            mbox-names = "tx";
            shmem = <0x03>;
            #address-cells = <0x01>;
            #size-cells = <0x00>;
            phandle = <0x18>;
        }
    }
}
```

```

        protocol@13 {
            reg = <0x13>;
            #clock-cells = <0x01>;
            phandle = <0x09>;
        };

        protocol@15 {
            reg = <0x15>;
            #thermal-sensor-cells = <0x01>;
            phandle = <0x04>;
        };
    };

    // optee {
    //     compatible = "linaro,optee-tz";
    //     method = "smc";
    // };
};

cpus {
    #address-cells = <0x02>;
    #size-cells = <0x00>;
    phandle = <0x19>;

    cpu-map {

        cluster0 {

            core0 {
                cpu = <0x05>;
            };
        };

        cluster1 {

            core0 {
                cpu = <0x06>;
            };
        };
    };

    cpu@100 {
        device_type = "cpu";
        compatible = "phytium,ftc664\0arm,armv8";
        reg = <0x00 0x00>;
        enable-method = "psci";
        clocks = <0x09 0x00>;
        capacity-dmips-mhz = <0x161c>;
        phandle = <0x05>;
    };

    cpu@101 {
        device_type = "cpu";
        compatible = "phytium,ftc664\0arm,armv8";
        reg = <0x00 0x100>;
    };
};

```

```

        enable-method = "psci";
        clocks = <0x09 0x01>;
        capacity-dmips-mhz = <0x161c>;
        phandle = <0x06>;
    };
};

interrupt-controller@30800000 {
    compatible = "arm,gic-v3";
    #interrupt-cells = <0x03>;
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    ranges;
    interrupt-controller;
    reg = <0x00 0x30800000 0x00 0x20000 0x00 0x30880000 0x00 0x80000 0x00
0x30840000 0x00 0x10000 0x00 0x30850000 0x00 0x10000 0x00 0x30860000 0x00
0x10000>;
    interrupts = <0x01 0x09 0x08>;
    phandle = <0x01>;

    // gic-its@30820000 {
    //     compatible = "arm,gic-v3-its";
    //     msi-controller;
    //     reg = <0x00 0x30820000 0x00 0x20000>;
    //     phandle = <0x0f>;
    // };
};

pmu {
    compatible = "arm,armv8-pmuv3";
    interrupts = <0x01 0x07 0x08>;
};

timer {
    compatible = "arm,armv8-timer";
    interrupts = <0x01 0x0d 0x08 0x01 0x0e 0x08 0x01 0x0b 0x08 0x01 0x0a
0x08>;
    clock-frequency = <0x2faf080>;
};

clocks {
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    ranges;
    clk50mhz {
        compatible = "fixed-clock";
        #clock-cells = <0x00>;
        clock-frequency = <0x2faf080>;
        phandle = <0x0d>;
    };
};

```

```

soc {
    compatible = "simple-bus";
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    dma-coherent;
    ranges;
    phandle = <0x1a>;
    uart@2800d000 {
        compatible = "arm,pl011\0arm,primecell";
        reg = <0x00 0x2800d000 0x00 0x1000>;
        interrupts = <0x00 0x54 0x04>;
        clocks = <0x0c 0x0c>;
        clock-names = "uartclk\0apb_pclk";
        status = "okay";
        phandle = <0x22>;
    };
    mailbox@32a00000 {
        compatible = "phytium,mbox";
        reg = <0x00 0x32a00000 0x00 0x1000>;
        interrupts = <0x00 0x16 0x04>;
        #mbox-cells = <0x01>;
        phandle = <0x02>;
    };

    sram@32a10000 {
        compatible = "phytium,pe220x-sram-ns\0mmio-sram";
        reg = <0x00 0x32a10000 0x00 0x2000>;
        #address-cells = <0x01>;
        #size-cells = <0x01>;
        ranges = <0x00 0x00 0x32a10000 0x2000>;
        phandle = <0x67>;

        scp-shmem@0 {
            compatible = "arm,scmi-shmem";
            reg = <0x1000 0x400>;
            phandle = <0x68>;
        };

        scp-shmem@1 {
            compatible = "arm,scmi-shmem";
            reg = <0x1400 0x400>;
            phandle = <0x03>;
        };
    };
};

// virtio blk
virtio_mmio@a003c00 {
    dma-coherent;
    interrupt-parent = <0x01>;
    interrupts = <0x0 0x2e 0x1>;
    reg = <0x0 0xa003c00 0x0 0x200>;
    compatible = "virtio,mmio";
    status = "okay";
};

```

```

//virtio serial
virtio_mmio@a003800 {
    dma-coherent;
    interrupt-parent = <0x01>;
    interrupts = <0x0 0x2c 0x1>;
    reg = <0x0 0xa003800 0x0 0x200>;
    compatible = "virtio,mmio";
};
chosen {
    stdout-path = "/virtio_mmio@a003800";
    bootargs = "clk_ignore_unused earlycon=virtio,mmio,0xa003800
console=hvc0 root=/dev/vda rootwait rw";
};
memory@b0000000 {
    device_type = "memory";
    reg = <0x0 0xb0000000 0x00 0x30000000>;
};

__symbols__ {
    scmi = "/firmware/scmi";
    scmi_dvfs = "/firmware/scmi/protocol@13";
    scmi_sensors0 = "/firmware/scmi/protocol@15";
    cpu = "/cpus";

    cpu_b0 = "/cpus/cpu@100";
    cpu_b1 = "/cpus/cpu@101";
    gic = "/interrupt-controller@30800000";
    sysclk_48mhz = "/clocks/clk48mhz";
    sysclk_50mhz = "/clocks/clk50mhz";
    sysclk_100mhz = "/clocks/clk100mhz";
    sysclk_200mhz = "/clocks/clk200mhz";
    sysclk_250mhz = "/clocks/clk250mhz";
    sysclk_300mhz = "/clocks/clk300mhz";
    sysclk_600mhz = "/clocks/clk600mhz";
    sysclk_1200mhz = "/clocks/clk1200mhz";
    soc = "/soc";
    uart1 = "/soc/uart@2800d000";
    mbox = "/soc/mailbox@32a00000";
    sram = "/soc/sram@32a10000";
    cpu_scp_lpri = "/soc/sram@32a10000/scp-shmem@0";
    cpu_scp_hpri = "/soc/sram@32a10000/scp-shmem@1";
};
};

```

3.4.3 Zone1 配置文件说明 (zone1-linux.json)

non-root zones的配置则存储在root linux的文件系统中，通常以JSON格式表示，zone1-linux.json：

```

{
    "arch": "arm64",
    "name": "linux2",
    "zone_id": 1,
    "cpus": [2,3],
    "memory_regions": [

```

```

{
    "type": "ram",
    "physical_start": "0xb0000000",
    "virtual_start": "0xb0000000",
    "size": "0x30000000"
},
{
    "type": "io",
    "physical_start": "0x2800d000",
    "virtual_start": "0x2800d000",
    "size": "0x1000"
},
{
    "type": "virtio",
    "physical_start": "0xa003c00",
    "virtual_start": "0xa003c00",
    "size": "0x200"
},
{
    "type": "virtio",
    "physical_start": "0xa003800",
    "virtual_start": "0xa003800",
    "size": "0x200"
}
],
"interrupts": [54, 76, 78, 116],
"ivc_configs": [],
"kernel_filepath": "./Image",
"dtb_filepath": "./linux2.dtb",
"kernel_load_paddr": "0xb0400000",
"dtb_load_paddr": "0xb0000000",
"entry_point": "0xb0400000",
"arch_config": {
    "gic_version": "v3",
    "gicd_base": "0x30800000",
    "gicd_size": "0x20000",
    "gicr_base": "0x30880000",
    "gicr_size": "0x80000",
    "gits_base": "0x0",
    "gits_size": "0x0"
}
}

```

- 该文件中包含如下关键字段：

- `zone_id`：标识 zone 编号，需唯一；
- `cpus`：指定分配给该虚拟机的物理 CPU 核心；
- `memory_regions`：描述 RAM 和 I/O 区域的地址映射；
- `interrupts`：该 zone 允许接收的中断号列表；
- `kernel_filepath` / `dtb_filepath`：Linux 内核与设备树的路径；
- `entry_point` / `kernel_load_paddr`：内核入口与加载地址；
- `arch_config`：定义中断控制器的地址、大小与版本（GICv3）。

该 JSON 文件最终被 `hvisor-tool` 解析为结构体后传递给 Hypervisor 用于创建对应 zone。

3.4.4 启动流程

1. 加载内核模块

启动命令行工具之前，首先需在 `zone0` 加载 `hvisor.ko` 内核模块：

```
insmod hvisor.ko
```

如果需要卸载模块，可使用：

```
rmmod hvisor.ko
```

2. 启动 Virtio 守护进程（可选）

若 `zone1` 需使用 Virtio 设备（如块设备、串口、网卡），需预先启动 Virtio 守护进程：

```
nohup ./hvisor virtio start zone1-linux-virtio.json &
```

其中 `zone1-linux-virtio.json` 描述了各个 Virtio 设备的类型、地址、镜像、IRQ 等参数。该守护进程会将 Zone 的内存映射到自身地址空间，模拟设备行为，并响应来自虚拟机的 MMIO 操作。

3. 启动 Zone1

Virtio 守护进程启动后，即可启动 `zone1`：

```
./hvisor zone start zone1-linux.json
```

该命令会将内核镜像和设备树加载至目标内存地址，并启动指定 CPU 执行。

3.4.5 Virtio 配置说明 (zone1-linux-virtio.json)

Virtio 配置文件中每个设备定义包含以下字段：

```
{
  "zones": [
    {
      "id": 1,
      "memory_region": [
        {
          "zone0_ipa": "0xb0000000",
          "zone1_ipa": "0xb0000000",
          "size": "0x30000000"
        }
      ],
      "devices": [
        {
          "type": "blk",
          "addr": "0xa003c00",
          "len": "0x200",
          "irq": 78,
          "img": "rootfs2.ext4",
          "status": "enable"
        },
        {
          "type": "console",
```

```

        "addr": "0xa003800",
        "len": "0x200",
        "irq": 76,
        "status": "enable"
    }
}
]
}
]
}

```

- **blk**: 配置块设备，映射磁盘镜像文件；
- **console**: 配置控制台输出设备；
- **net (可选)** : 配置虚拟网卡设备，连接到 Tap 接口；

所有 Virtio 设备会在用户态完成 MMIO 区域模拟，并通过共享内存与虚拟机通信。

3.4.6 完整示例：在飞腾派上启动 Zone1

以下命令用于在实际平台（如 Phytium-Pi）上挂载必要文件系统并启动 `zone1`：

```

mount -t proc proc /proc
mount -t sysfs sysfs /sys
mkdir -p /dev/pts
mount -t devpts devpts /dev/pts
rm nohup.out

nohup ./hvisor virtio start zone1-linux-virtio.json &
./hvisor zone start zone1-linux.json

```

确保内核镜像、设备树和镜像文件位于 `zone0` 文件系统中，且 `zone1` 使用的资源未与 `zone0` 冲突。

```

[ 11.530996] hub 5-0:1.0: 1 port detected
[ 11.535232] phyti-otg 32840000.usb2: Phytium Host USB Driver
[ 11.541067] phyti-otg 32840000.usb2: new USB bus registered, assigned bus number 6
[ 11.549105] hub 6-0:1.0: USB hub found
[ 11.552895] hub 6-0:1.0: 1 port detected
[ 11.557139] i2c /dev entries driver
[ 11.562529] sbsa-gwtd 28041000.watchdog: Initialized with 30s timeout @ 50000000 Hz, action=0.
[ 11.571467] sbsa-gwtd 28043000.watchdog: Initialized with 30s timeout @ 50000000 Hz, action=0.
[ 11.580271] Bluetooth: HCI UART driver ver 2.3
[ 11.584719] Bluetooth: HCI UART protocol Three-wire (H5) registered
[ 11.591284] sdhci: Secure Digital Host Controller Interface driver
[ 11.597519] sdhci: Copyright(c) Pierre Ossman
[ 11.601932] Synopsys Designware Multimedia Card Interface Driver
[ 11.608101] sdhci-pltfm: SDHCI platform and OF driver helper
[ 11.614010] phyti-mci-platform 28000000.mmc: mci clk set 0 0 0x0 0x0 0x0 0x0
[ 11.621297] phyti-mci-platform 28000000.mmc: init hardware done!
[ 11.652608] phyti-mci-platform 28000000.mmc: phyti_mci_probe 114: probe phyti mci successful.
[ 11.661875] phyti-mci-platform 28001000.mmc: mci clk set 0 0 0x0 0x0 0x0 0x0
[ 11.669258] phyti-mci-platform 28001000.mmc: init hardware done!
[ 11.680844] phyti-mci-platform 28000000.mmc: card claims to support voltages below defined range
[ 11.689805] phyti-mci-platform 28000000.mmc: no support for card's volts
[ 11.696709] mmc0: error -22 whilst initialising SDIO card
[ 11.702105] phyti-mci-platform 28001000.mmc: phyti_mci_probe 114: probe phyti mci successful.
[ 11.711572] ledtrig-cpu: registered to indicate activity on CPUs
[ 11.716697] phyti-mci-platform 28001000.mmc: card claims to support voltages below defined range
[ 11.717917] usbcore: registered new interface driver usbhid
[ 11.732110] usbhid: USB HID core driver
[ 11.736200] phyti-mbox 32a00000.mailbox: Phytium SoC Mailbox registered
[ 11.740800] mmc1: new high speed SDIO card at address 0001
[ 11.743453] usbcore: registered new interface driver snd-usb-audio
[ 11.755027] genirq: irq chip 28036000.gpio did not update eff. affinity mask of irq 100
[ 11.762924] mmc0: host does not support reading read-only switch, assuming write-enable
[ 11.771978] NET: Registered protocol family 17
[ 11.775220] mmc0: new high speed SDXC card at address aaaa
[ 11.776492] 9pnet: Installing 9P2000 support
[ 11.782144] mmcblk0: mmc0:aaaa SD64G 59.5 GiB
[ 11.786183] Key type dns_resolver registered
[ 11.795044] registered taskstats version 1
[ 11.799140] Loading compiled-in X.509 certificates
[ 11.801051] mmcblk0: p1
[ 11.804350] arm-scmi firmware:scmi: SCMI Notifications - Core Enabled.
[ 11.813152] arm-scmi firmware:scmi: SCMI Protocol v2.0 'Phytium:E2000' Firmware version 0x2050000
[ 11.942361] cpu cpu0: EM: created perf domain
[ 11.947075] cpu cpu3: EM: created perf domain
[ 11.952072] of_cfs_init
[ 11.954668] of_cfs_init: OK
[ 11.957519] clk: Not disabling unused clocks
[ 11.961841] ALSA device list:
[ 11.964799] #0: phyti.pe220x-i2s-audio
[ 11.968962] uart-pl011 2800d000.uart: no DMA platform data
[ 12.007290] EXT4-fs (mmcblk0p1): recovery complete
[ 12.013204] EXT4-fs (mmcblk0p1): mounted filesystem with ordered data mode. Opts: (null)
[ 12.021313] VFS: Mounted root (ext4 filesystem) on device 179:1.
[ 12.040166] devtmpfs: mounted
[ 12.043954] Freeing unused kernel memory: 2432K
[ 12.048528] Run /bin/sh as init process
/bin/sh: 0: can't access tty; job control turned off
# bash
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@(none):/# mount -t proc proc /proc
root@(none):/# mount -t sysfs sysfs /sys
root@(none):/# cat /proc/cpuinfo
processor       : 0
BogoMIPS      : 100.00
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 cpuid sha3 sha512
CPU implementer : 0x70
CPU architecture: 8
CPU variant    : 0x1
CPU part       : 0x664
CPU revision   : 5

processor       : 3
BogoMIPS      : 100.00
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 cpuid sha3 sha512
CPU implementer : 0x70
CPU architecture: 8
CPU variant    : 0x1
CPU part       : 0x664
CPU revision   : 5
root@(none):/#

```

non-root linux使用CPU2、CPU3

```
[WARN 1] (hvisor::device::irqchip:64) MMIO init for irqchip
[WARN 1] (hvisor::device::irqchip:73) MMIO init for GICv3
[INFO 1] (hvisor::device::irqchip::gicv3::vgic:53) registering gicr cpu0 at 0x30880000
[INFO 1] (hvisor::device::irqchip::gicv3::vgic:53) registering gicr cpu1 at 0x308a0000
[INFO 1] (hvisor::device::irqchip::gicv3::vgic:53) registering gicr cpu2 at 0x308c0000
[INFO 1] (hvisor::device::irqchip::gicv3::vgic:53) registering gicr cpu3 at 0x308e0000
[INFO 1] (hvisor::device::irqchip::gicv3::vgic:67) Found interrupt in Zone 1 irq_bitmap: 76
[INFO 1] (hvisor::device::irqchip::gicv3::vgic:67) Found interrupt in Zone 1 irq_bitmap: 78
[INFO 1] (hvisor::zone:265) zone cpu set: 0b1100
[INFO 2] (hvisor::event:121) cpu 2 wakeup
00:47 INFO[INFO 2] (hvisor::arch::aarch64::cpu:195) cpu 2 started at 0xb0400000
hvisor.c:532: Zone name: linux2
08:00:47 WARN safe_cjson.c:24: hvisor.c:172 - [cJSON_GetObjectItem] Key 'gicc_base' not found in JSON object
08:00:47 WARN safe_cjson.c:24: hvisor.c:174 - [cJSON_GetObjectItem] Key 'gich_base' not found in JSON object
08:00:47 psci smc call: code=0x84000000, arg0=0x0
psci smc call: code=0x84000006, arg0=0x0
psci smc call: code=0x8400000a, arg0=0x80000000
psci smc call: code=0x8400000a, arg0=0xc4000001
psci smc call: code=0x8400000a, arg0=0xc400000e
psci smc call: code=0x8400000a, arg0=0xc4000012
WARN safe_cjson.c:24: hvisor.c:176 - [cJSON_GetObjectItem] Key 'gicv_base' not found in JSON object
08:00:47 WARN safe_cjson.c:24: hvisor.c:178 - [cJSON_GetObjectItem] Key 'gicc_offset' not found in JSON object
08:00:47 WARN safe_cjson.c:24: hvisor.c:180 - [cJSON_GetObjectItem] Key 'gicv_size' not found in JSON object
08:00:47 WARN psci smc call: code=0xc4000003, arg0=0x100
[INFO 2] (hvisor::arch::aarch64::trap:343) psci: try to wake up cpu 3
psci: try to wake up cpu 3
[[90msafe_cjson.c:INFO 3] (hvisor::event:121) cpu 3 wakeup
24: hvisor.[37m[INFO 3] (hvisor::arch::aarch64::cpu:195) cpu 3 started at 0xb10ff1dc
c:182 - [cJSON_GetObjectItem] Key 'gich_size' not found in JSON object
08:00:47 WARN safe_cjson.c:24: hvisor.c:184 - [cJSON_GetObjectItem] Key 'gicc_size' not found in JSON object
08:00:47 WARN safe_cjson.c:24: hvisor.c:276 - [cJSON_GetObjectItem] Key 'pci_config' not found in JSON object
08:00:47 WARN hvisor.c:278: No pci_config field found.
08:00:47 INFO hvisor.c:555: Calling ioctl to start zone: [linux2]
root@none:/home# ./hvisor zone list
  zone_id |      cpus |      name |      status |
  ---|---|---|---|
    0 | 0, 1 | root-linux | running |
    1 | 2, 3 | linux2 | running |
root@none:/home# ./hvisor zone shutdown -id 1
[INFO 0] (hvisor::hypercall:282) handle hvc zone shutdown id 1
[INFO 3] (hvisor::arch::aarch64::cpu:233) cpu 3 start parking
[INFO 0] (hvisor::hypercall:341) zone 1 has been shutdown
[INFO 2] (hvisor::arch::aarch64::cpu:233) cpu 2 start parking
root@none:/home# ./hvisor zone list
  zone_id |      cpus |      name |      status |
  ---|---|---|---|
    0 | 0, 1 | root-linux | running |
root@none:/home#
```

成功启动两个虚拟机

执行关闭zone1虚拟机命令

成功关闭zong1虚拟机

3.4.7 关闭 Zone 与 Virtio 设备

关闭指定 Zone 的命令：

```
./hvisor zone shutdown -id 1
```

列出所有运行中的 Zone：

```
./hvisor zone list
```

关闭所有 Virtio 设备及守护进程：

```
kill hvisor-virtio
```

四、决赛阶段计划

1. 完成hvisor对翰博微E2000Q教育开发板的适配。
2. 完成基于共享内存和核间中断的虚拟机间通信机制的设计HyperAMP。
3. 将sel4移植到飞腾派平台。
4. sel4作为 Guest OS 行于 hvisor 的 Non-Root虚拟机中，并能通过HyperAMP读取到root linux写入共享内存中的数据。
5. 进行性能测试和优化。

